

ShopSense: Intelligent E-Commerce Platform

by

Arifa Khatun | **Md.Sahedur Rahman**
213002176 | **213002140**

*Project report (CSE 400) submitted in partial fulfillment of the
requirements for the degree of*

Bachelor of Science in Computer Science and Engineering

Under the supervision of

Wahia Tasnim
Lecturer, Dept. of Artificial Intelligence and Data Science



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GREEN UNIVERSITY OF BANGLADESH

FALL 2025



© *Arifa Khatun and Md.Sahedur Rahman*
All rights reserved

DECLARATION

Title	ShopSense: Intelligent E-Commerce Platform
Authors	Arifa Khatun and Md.Sahedur Rahman
Student IDs	213002176 and 213002140
Supervisor	Wahia Tasnim

We declare that this project report entitled *ShopSense: Intelligent E-Commerce Platform* is the result of our own work except as cited in the references. The project report has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

Arifa Khatun
213002176

Department of Computer Science and Engineering
Green University of Bangladesh

Md.Sahedur Rahman
213002140

Department of Computer Science and Engineering
Green University of Bangladesh

Date: January 30, 2026

CERTIFICATE OF ENDORSEMENT

This is to certify that the project report entitled *ShopSense: Intelligent E-Commerce Platform* by **Arifa Khatun** (Student ID: 213002176), and **Md.Sahedur Rahman** (Student ID: 213002140) has been successfully defended and examined.

Based on the evaluation and discussion held on **January 30, 2026**, we, the undersigned, hereby endorse the project report for acceptance and approval.

Md. Riaz Uddin
External Member, Board: 10

Assistant Maintenance Engineer
Ministry of Agriculture

Montaser Abdul Quader
Board Member, Board: 10

Lecturer
Department of Computer Science and Engineering
Green University of Bangladesh

Ms. Farhana Akter Sunny
Chair, Board: 10

Assistant Professor
Department of Computer Science and Engineering
Green University of Bangladesh

Date: January 30, 2026



Department of Computer Science and Engineering
Green University of Bangladesh
Kanchon, Rupganj, Narayanganj - 1461

CERTIFICATE

This is to certify that the project report entitled **ShopSense: Intelligent E-Commerce Platform**, submitted by **Arifa Khatun** (Student ID: 213002176) and **Md.Sahedur Rahman** (Student ID: 213002140) are undergraduate students of the **Department of Computer Science and Engineering** has been examined. Upon recommendation by the examination committee, we hereby accord our approval of it as the presented work and submitted report fulfill the requirements for its acceptance in partial fulfillment for the degree of *Bachelor of Science in Computer Science and Engineering*.

Wahia Tasnim
Lecturer, Dept. of Artificial Intelligence and Data Science

Department of Computer Science and Engineering
Green University of Bangladesh

Syed Ahsanul Kabir
Professor and Chairperson of Department of CSE

Department of Computer Science and Engineering
Green University of Bangladesh

Place: Narayanganj

Date: January 30, 2026

ACKNOWLEDGMENTS

We would like to express our deep and sincere gratitude to our supervisor, *Wahia Tasnim*, for allowing us to conduct research and providing invaluable guidance throughout this work. Her dynamism, vision, sincerity and motivation have deeply inspired us. She has taught us the methodology to carry out the work and to present the works as clearly as possible. It was a great privilege and honour to work and study under her guidance.

We are greatly indebted to our honourable teachers of the Department of Computer Science and Engineering at the Green University of Bangladesh who taught us during our study. Without any doubt, their teaching and guidance have completely transformed us to the persons that we are today.

We are extremely thankful to our parents for their unconditional love, endless prayers and caring, and immense sacrifices for educating and preparing us for our future. We would like to say thanks to our friends and relatives for their kind support and care.

Finally, we would like to thank all the people who have supported us to complete the project work directly or indirectly.

Arifa Khatun and Md.Sahedur Rahman

Green University of Bangladesh

Date: January 30, 2026



Dedicated to
my loving Mother

– *Arifa Khatun*

To my beloved parents
the source of my inspiration

– *Md.Sahedur Rahman*

ABSTRACT

In years online shopping has become a part of everyday life. People shop online every day. Online shopping now changes the way people interact with stores and products. ShopSense: Intelligent E-Commerce Platform is a web based system. It mixes an store, with Artificial Intelligence (AI) and Machine Learning (ML). It makes product discovery and purchasing smoother and more personal. It uses AI to suggest items that match the shopper's taste. It learns from each purchase to improve recommendations. Online shopping with ShopSense: Intelligent E-Commerce Platform feels like a talk, with a friend. The system offers a complete shopping pipeline, including dynamic product and category management, advanced search, AI-driven recommendations, shopping cart, secure multi-payment checkout, and order tracking. ShopSense is organized into three main components: a customer-facing storefront for browsing products, interacting with an AI chatbot, and placing orders; an intelligent backend layer that powers ML-based semantic search, recommendation algorithms, and analytics; and an admin panel for managing catalog data, coupons, orders, and customer interactions. The platform is implemented using Django and REST APIs on the backend, with integrated payment gateways (such as Stripe and bKash) and a relational database for reliable data persistence. ML models are used to generate product embeddings, track user interactions, and rank search results to deliver more relevant and personalized product lists. By reducing friction in product discovery, automating repetitive support queries through a chatbot, and providing richer insights into customer behavior, ShopSense improves user satisfaction and supports store owners in scaling their online business. This project contributes to the evolving field of AI-enhanced e-commerce by demonstrating how web technologies and ML techniques can be combined to build an intelligent, data-driven online shopping platform.

Keywords: E-commerce, Intelligent E-Commerce Platform, Conversational AI, Machine Learning, Product Recommendation, Semantic Search, Online Shopping Cart, Order Management

Contents

1	Introduction	1
1.1	Introduction	1
1.2	Motivation	2
1.3	Objectives of the Project	3
1.4	Project Scope	4
1.5	Project Contribution	5
1.6	Gantt Chart	6
1.7	Budget Details	7
1.8	Organization of the Report	7
1.9	Conclusion	8
2	Literature Review	9
2.1	Introduction	9
2.2	Related Works on E-Commerce and AI	9
2.2.1	Overview of E-Commerce Platforms	9
2.2.2	Web Frameworks for E-Commerce (Django, Node, Laravel, etc.)	10
2.2.3	AI in E-Commerce	11
2.2.3.1	Chat-bots in Online Retail	11
2.2.3.2	Semantic Search in Product Catalogs	12
2.2.3.3	Recommendation Systems (Content-based, Collaborative)	13
2.2.4	Online Payment Integration	14
2.2.4.1	Stripe and Global Card Payments	14
2.2.4.2	bKash and Mobile Payments in Bangladesh	15
2.3	Summary of Reviewed Work and Identified Gaps	16
2.4	Conclusion	18

3	Methodology and System Design	19
3.1	Introduction	19
3.2	Software Process Model and Methodology	19
3.3	System Requirements	20
3.3.1	Functional Requirements	20
3.3.2	Non-Functional Requirements (Performance, Security, Usability)	22
3.4	System Design and Architecture	24
3.4.1	High-Level Architecture Diagram (Django + Vue + APIs + DB)	24
3.4.2	Data Flow Between Front-end, Backend, and AI Services	25
3.5	Backend Design (Django)	27
3.5.1	Database Schema and Models (Products, Users, Orders, Cart, Coupons)	28
3.5.2	REST API Endpoints (Cart API, Product API, Recommendations API, Coupon API)	29
3.6	Frontend Design (Vue.js)	30
3.6.1	Component Structure (Home, Catalog, Product Detail, Cart, Checkout, Profile)	31
3.6.2	State Management (Vuex) and Routing	32
3.7	AI Components	32
3.7.1	Chatbot Architecture (LangChain + Ollama/OpenAI + ChromaDB)	32
3.7.2	Semantic Search System (Sentence Transformers, Vector Index)	34
3.7.3	Recommendation Engine	34
3.8	Payment System Design	36
3.8.1	Stripe Integration Flow	36
3.8.2	bKash Integration Flow (Tokenized Checkout)	38
3.8.3	Mock Payment Gateway for Testing	40
3.9	Security and Performance Considerations	42
3.9.1	Authentication, Authorization, CSRF	42
3.9.2	Static File Optimization, Caching, Database Tuning	42
3.10	Comparative Analysis with Existing Platforms	43
3.11	Engineering Considerations	45
3.12	Conclusion	49

4	Implementation	50
4.1	Introduction	50
4.2	Development Environment and Tech Stack	50
4.2.1	Backend Tools (Django, Python Libraries)	50
4.2.2	Frontend Tools (Vue 3, Vuex, TailwindCSS)	52
4.2.3	AI/ML Libraries (LangChain, ChromaDB, Sentence Trans- formers, scikit-learn)	53
4.3	User Interface Screenshots	54
4.3.1	Customer-Facing Views	54
4.3.2	AI Features: Chatbot, Search, and Recommendations	60
4.3.3	Administrative and Responsive Views	62
4.4	Core E-Commerce Features	64
4.4.1	Product Management and Catalog	64
4.4.2	Shopping Cart and Session Handling	64
4.4.3	Checkout and Order Processing	65
4.4.4	Coupon System Implementation	66
4.5	AI Features Implementation	67
4.5.1	Chatbot Integration in the Web UI	67
4.5.2	Product Ingestion and Vector Store Creation	68
4.5.3	Recommendation Algorithms and Pipelines	69
4.6	Payment Integration Implementation	70
4.6.1	Stripe API Keys, Webhooks, and Payment Flow	70
4.6.2	bKash Sandbox and Live Configuration	71
4.6.3	Error Handling and Transaction Logging	71
4.7	Deployment Setup	72
4.7.1	Environment Variables and Configuration	72
4.8	Testing and Validation	73
4.8.1	Test Strategy and Methodology	73
4.8.2	Module Testing with Test Cases	74
4.8.3	System Comparison with Existing Platforms	82
4.8.4	Deployment on Render	86
4.8.5	Static Files and Media Handling in Production	86
4.9	Conclusion	87

5	Conclusion and Future Work	88
5.1	Conclusion	88
5.2	General Discussion	90
5.3	Limitations of the Project	91
5.4	Future Work	93
5.5	Practical Implications	94
	References	96

List of Figures

1.1	ShopSense project timeline showing the main development phases over a 16-week period.	6
2.1	ShopSense targets the identified gaps and existing e-commerce platforms in this conceptual overview.	17
3.1	High-level architecture of the ShopSense platform, in which interactions among the Vue.js front end, Django backend, relational database, AI services, and payment gateways are visualized.	25
3.2	Data flow between the Vue.js frontend, Django REST API, AI components (semantic search, recommendations, chatbot), and the relational database.	27
3.3	Entity–relationship diagram of the ShopSense database schema, including users, products, categories, carts, orders, order items, and coupons.	28
3.4	Simplified Vue.js component structure for the ShopSense frontend.	31
3.5	Architecture of the ShopSense AI chatbot using retrieval-augmented generation with ChromaDB and configurable LLM backends.	33
3.6	Semantic search architecture using sentence transformers and ChromaDB for vector similarity retrieval.	34
3.7	Stripe payment integration flow for card payments in ShopSense.	38
3.8	bKash checkout flow in ShopSense.	40
3.9	Mock payment gateway flow used for development and testing.	41
4.1	ShopSense homepage with navigation bar, search entry point, and featured products.	54
4.2	Catalog page showing product cards with filtering and sorting options.	55

4.3	Single product detail view with images, description, and purchase actions.	56
4.4	Shopping cart page listing selected items, quantities, subtotal, and coupon input.	57
4.5	Checkout page including address details, payment method choice, and order summary.	57
4.6	Payment methods page showing Stripe card payment integration (other methods selectable from the same flow).	58
4.7	Order success page with confirmation of items, total cost, and payment status.	59
4.8	User profile page showing account details and links to order history.	59
4.9	AI chatbot interface assisting a user with product-related queries. . .	60
4.10	Search results page showing products retrieved via semantic search.	61
4.11	Product detail page with AI-powered recommendations section. . . .	61
4.12	Django admin dashboard used to manage products, coupons, and orders.	62
4.13	Mobile view of ShopSense showing responsive layout and navigation.	63

List of Tables

1.1	ShopSense project budget summary.	7
3.1	Overview of key REST API endpoints in the ShopSense backend. . .	30
3.2	Comparison of ShopSense with existing e-commerce platforms. . . .	44
4.1	Test cases for User Authentication Module	75
4.2	Test cases for Product Catalog Module	76
4.3	Test cases for Cart Management Module	77
4.4	Test cases for Checkout and Payment Module	78
4.5	Test cases for Semantic Search Module	79
4.6	Test cases for Recommendation Engine	80
4.7	Test cases for AI Chatbot	81
4.8	Test cases for Order Management Module	82
4.9	Implementation comparison: ShopSense vs existing platforms	83
4.10	Overall test coverage summary by module	85

1. Introduction

1.1 Introduction

In last few years, growth in web technologies, Artificial Intelligence (AI), and Machine Learning (ML), with the availability of the Internet, have changed how people online shopping for products . Previously shopping is often limited by store location, inventory constraints, and long communication between customers and sales staff. Now a days, modern consumers increasingly expect convenience, personalized recommendations, and quick access to relevant products across a wide range of categories.

Bangladesh's uprising bond with the global retail and technology sector, with increasing local interest in virtual-online businesses, has created a Desire for scalable e-commerce solutions. However, most of the small and medium-sized businesses still not have the technical infrastructure to build their own online e-commerce platforms. They mainly depends on basic web shops or social media pages that do not provide advanced search, recommendation, or analytics capabilities. As a result, customers find it hard to quickly find suitable products, compare alternatives, or receive meaningful guidance without going to the physical stores.

Some research on online retail point out that seamless digital interactions and reduced friction while browsing and checkout are critical for retaining customers and improving sales performance[?]. On the other hand, other researches on digital and mobile financial services point out the importance of offering multiple secure and convenient payment options such as bank cards and mobile wallets(MFS) to increase user trust and promote online transaction adoption in developing countries [?]. In Bangladesh, mobile financial services like bKash have become widely accepted, yet many smaller e-commerce sites still not willing to integrate these services into their sites for a unified, automated shopping experience.

To resolve these challenges and problems, our project introduces *ShopSense: Intelligent E-Commerce Platform*, an AI-enhanced online shopping system that solves the challenges and provide a traditional e-commerce workflow with intelligent search, recommendation, and chat support. The platform helps customers to browse products, apply filters, ML-driven recommendations, interact with the chat-bot(RAG), manage their cart, complete secure payments, and track orders. Admin/Shop-Owner can manage products, categories, coupons, and orders using dedicated backend interface, supported by analytics derived from user interactions.

ShopSense is developed using Django for the backend, HTML/CSS and modern web tools for the front-end, and a relational database for a reliable data storage. AI and ML components are used to generate product embeddings, power semantic search, and uses user behavior to continuously improve recommendation and user experience. By combining these technologies, the system aims to reduce traditional processes, improve product relevance, support multiple payment system (including local gateways), and provide a better, data-driven shopping experience for customers and store owners.

1.2 Motivation

In this growing digital commerce environment, many small and medium-sized online stores still depends on partially manual processes for order placement and payment. For example, platforms like ghorerbazar.com require or prefer customers to contact the seller manually (e.g., phone calls or messaging apps) to confirm orders and mainly support Cash on Delivery (COD) as the payment method. This process can introduce some friction, delays, and a risk of miscommunication, and it can make the ability to scale or automate business operations and need manpower to operate smoothly.

Few previous studies show that each additional click or interruption in the checkout process can increase users leaving and possibly reduce user satisfaction [?]. When customers are kind of forced to leave the website to complete an order or payment arrangement, they are more likely to abandon the purchase or switch to competitors that provide a seamless end-to-end flow [?, ?]. Only cash-on-delivery can make logistics more complex, and there is a higher probability of refusing deliv-

eries, which can cause small businesses to collapse as they have limited capacity for operations[?].

Supporting multiple payment options, such as card payments or mobile financial services, is proven by research to convince people to adopt online payments in developing economies [?]. In markets like Bangladesh, mobile financial services, such as bKash, have already become a part of everyday financial activity, and integrating them directly into e-commerce platforms can substantially increase user trust and transaction volume [?]. Platforms that rely only on manual payments or cash on delivery (COD) may fail to take advantage of these developments and remain vulnerable to inefficiencies and loss of revenue [?]. In this project, we will attempt to develop a system that can reduce manual steps, introduce multiple payment methods, and bring artificial intelligence (AI) to enhance customer service and product discovery while remaining straightforward and deployable for Bangladeshi small and medium-sized businesses.

1.3 Objectives of the Project

Our main goal for this project is to create and deploy an intelligent e-commerce platform that offers a seamless online shopping experience for both customers and store administrators. In order to accomplish this overall goal, the project aims to meet the following specific objectives:

- To construct a full-featured e-commerce web application that allows product browsing, cart management, checkout, and order tracking within a single online interface.
- To include several payment alternatives, such as card payments (via Stripe), mobile banking (via bKash), to eliminate manual management and cash-on-delivery only workflows [?].
- To design and deploy AI-based semantic search that allows customers to find relevant products using natural language queries and similarity-based retrieval;
- To implement a recommendation engine that provides personalized product suggestions based on product content and user interaction history;

- To build an AI chatbot that can answer product-related queries, remember context, assist with order information, and derive customers through the shopping process;
- To provide an admin backend for managing products, categories, coupons, users, and orders, along with basic analytics derived from transactional and interaction data.

1.4 Project Scope

The scope of this project is intentionally focused on building a practical, deployable prototype of an AI-enhanced e-commerce platform rather than a complete commercial system. The major components included within the scope are:

- Design and implementation of a web-based storefront where customers can register, log in, browse products by category, view details, manage a shopping cart, and place orders;
- Development of core backend services in Django, including models for products, categories, users, orders, payments, and coupons, as well as RESTful APIs for frontend integration;
- Integration of Stripe and bKash payment gateways in sandbox or test mode, alongside a mock payment gateway for development and demonstration purposes;
- Implementation of AI and ML components for semantic search, product recommendations, and chatbot assistance using appropriate libraries and tools;
- Creation of an admin dashboard for order processing, promotions, and catalog data management utilizing Django's admin panel and/or custom views.
- A simple deployment setup that will provide information about utilizing a cloud-friendly configuration, such as Render, Railway, or a comparable platform.

Some elements are outside the scope of this project, including:

- Complex multi-vendor marketplace features, such as seller portals or revenue sharing;
- Full-scale production hardening, such as extensive security audits, high-availability infrastructure, or large-scale performance optimization.

1.5 Project Contribution

In the context of intelligent commercial systems, this project makes various methodological and practical contributions:

- It shows how various payment methods, such as credit/debit cards, mobile banking, and COD, can be integrated to reduce checkout friction and encourage online transaction adoption in developing countries.
- It presents a unified architecture that combines a traditional online-commerce workflow with AI-driven search, recommendations, and conversational assistance tailored for small and medium-sized merchants [?, ?].
- It demonstrates a practical chatbot integration that uses retrieval-augmented generation to answer product and order-related queries naturally and contextually.
- It offers documentation, configuration guidelines, and example datasets helpful for students, researchers, and developers exploring AI-enhanced e-commerce applications.
- It offers a usable implementation of semantic search and recommendation features adaptable to similar retail domains.

The project shows how interaction and transaction data can be formatted to facilitate analytics, such as identifying popular products, user behavior trends, and promotion efficacy, in addition to these technological contributions [?].

1.6 Gantt Chart

The development of ShopSense was planned over several phases spanning approximately sixteen weeks, from early March to late October. The high-level timeline is illustrated in Figure 1.1, which shows the main phases of work and their approximate durations:

- **Task 1: Planning & Setup (Mar – May):** Requirements analysis and literature review, and system design.
- **Task 2: Core Development (May – Jul):** Implementation of authentication, user accounts, and product features.
- **Task 3: Advanced Features (Jun – Sep):** Implementation of checkout, order management, and payment integration.
- **Task 4: AI/ML Implementation (Jul – Oct):** Development of semantic search, recommendations, and chat-bot integration.
- **Task 5: Testing & Deployment (Sep – Nov):** System testing, bug fixing, performance checks, and deployment configuration.
- **Task 6: Documentation (Jun – Nov):** Documenting of each feature for project report.

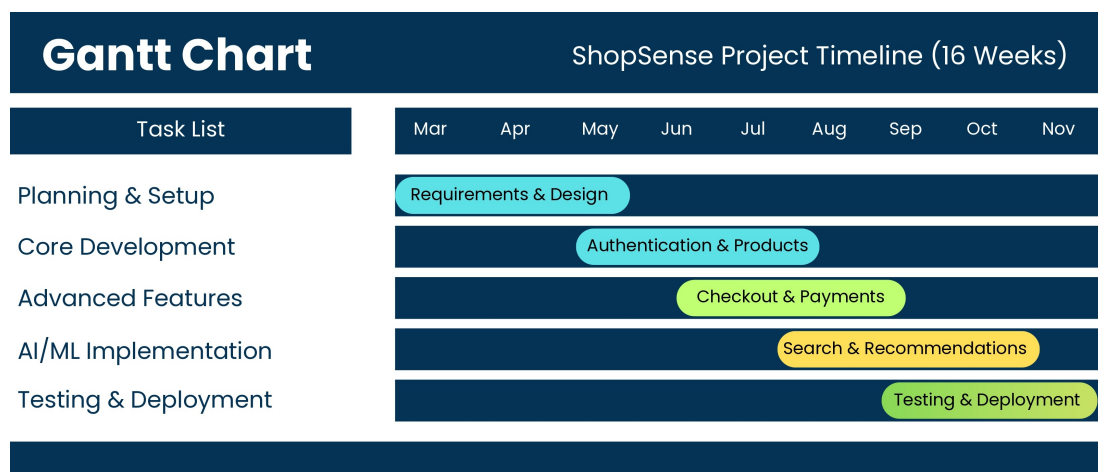


Figure 1.1: ShopSense project timeline showing the main development phases over a 16-week period.

1.7 Budget Details

Table 1.1 shows the projected sum for the ShopSense project, calculated in Bangladeshi Taka (BDT). The budget considers human resources, software and tooling, hosting, payment gateway setup, and other supporting costs, as well as a contingency allocation.

Table 1.1: ShopSense project budget summary.

Category	Cost (BDT)
Human Resources / (Developer-Designer cost)	62,000
Software & Tools	3,500
Cloud & Hosting	14,000
Payment Gateway Setup	5,000
Domain & SSL	2,000
Hardware & Equipment	5,500
Miscellaneous Expenses	6,900
Contingency (10%)	8,900
Total Estimated Cost	97,900

The budget assumes moderate-scale development effort with a small team, reliance on mostly open-source technologies, and usage of affordable cloud hosting options suitable for academic or pilot deployments.

1.8 Organization of the Report

The remainder of this report is organized as follows:

- **Chapter 2** presents the literature review and related work on e-commerce platforms, AI-driven search and recommendation systems, online payment integration, and mobile financial services in developing countries.
- **Chapter 3** describes the design methods and procedures, including the overall architecture of ShopSense, database schema, major system modules, data flow diagrams, and design considerations for AI and payment components.
- **Chapter 4** explains the implementation details of the ShopSense platform, covering backend and frontend development, integration of AI services, con-

figuration of payment gateways, deployment setup, and user interface screenshots.

- **Chapter 5** concludes the report by summarizing the main contributions, discussing general findings, outlining limitations of the current prototype, and presenting directions for future work and practical implications.

1.9 Conclusion

This chapter introduced the background, motivation, objectives, scope, and contributions of the *ShopSense* project, and outlined the structure of the remaining chapters. It also presented an overview of the project schedule and budget. The next chapter reviews related work in intelligent e-commerce, AI-driven search and recommendation, and online payment integration to position ShopSense within the broader research and industry context.

2. Literature Review

2.1 Introduction

This chapter reviews the existing literature and practice related to e-commerce platforms, web frameworks for custom online stores, applications of AI in e-commerce, and online payment integration with a focus on developing-country contexts such as Bangladesh. The aim is to identify strengths and limitations of current solutions, highlight open challenges, and motivate the need for an intelligent, AI-enhanced e-commerce platform such as *ShopSense*.

2.2 Related Works on E-Commerce and AI

2.2.1 Overview of E-Commerce Platforms

Electronic commerce (e-commerce) platforms provide the digital infrastructure that enables businesses to present products, manage inventory, process orders, and handle payments over the Internet [?]. Modern e-commerce platforms typically offer a combination of frontend storefront interfaces, backend administration panels, and integration points with third-party services such as payment gateways, logistics providers, and analytics tools. Popular commercial solutions include Software-as-a-Service (SaaS) platforms (e.g., Shopify, BigCommerce) and self-hosted content management systems (CMS) with e-commerce extensions (e.g., WooCommerce on WordPress, Magento/OpenCart) [?].

Already existing SaaS platforms provide services that include managed hosting, integrated payment interfaces, and pre-made templates to simplify initial setup. However, they used to restrict data control, customization, and the incorporation of new AI features or newer methods of payment. Although self-hosted and open-source platforms offer more freedom and flexibility, their deployment, security,

and maintenance require more technical expertise. The difference between cost, flexibility, and necessary technical skills is not suitable for small businesses country like Bangladesh as they're underdevelopment.

Many merchants in Bangladesh and similar countries still rely on semi-formal or informal online sales methods (such social media accounts) rather than fully integrated e-commerce websites. These systems usually lack integrated local payment options, intelligent recommendations, extensive search capabilities, and systematic data collection for analytics. Consequently, merchants have limited visibility into customer behavior and demand patterns, while customers have friction in product search and checkout [?, ?].

2.2.2 Web Frameworks for E-Commerce (Django, Node, Laravel, etc.)

Web application frameworks are essential for creating customized e-commerce platforms. They offer abstractions for data models, user authentication, template rendering, routing, and API development. The following are some of the most popular frameworks for commercial development:

- **Django (Python):** A high-level web framework that promotes quick development and clear, practical design[?]. Django is ideal for creating data-intensive applications like online stores because it offers a strong Object-Relational Mapping (ORM) layer, an authentication mechanism, and an automatically generated admin interface.
- **Node.js/Express and Related Stacks:** JavaScript-based Frontend frameworks like React, Vue, or Angular are frequently used in conjunction with JavaScript-based backend like Express and NestJS.
- **Laravel (PHP):** A feature-rich MVC framework with built-in authentication, beautiful syntax, and robust ecosystem support. Due to extensive hosting support, many small and medium-sized e-commerce sites are developed using Laravel or similar PHP frameworks.

Django is particularly attractive for research-oriented and AI-enhanced e-commerce projects because of Python's rich ecosystem of data science and machine learning libraries [?]. Python's robust ecosystem of data science and machine learning

libraries makes Django especially appealing for research-focused and AI-enhanced e-commerce enterprises. Frameworks like Django REST Framework (DRF) make it easier to create RESTful APIs, enabling front-end clients (such as Vue.js applications) and external AI services.

2.2.3 AI in E-Commerce

Chat-bots, semantic search, and recommendation systems are the three main online application areas of AI that are pertinent to this project. AI technologies have become integrated to better e-commerce platforms, enabling more effective product discovery, tailored user experiences, and smarter customer care.

2.2.3.1 Chat-bots in Online Retail

Chatbots are conversational agents that use natural language to communicate with customers, usually through messaging apps or web chat widgets[?]. Chatbots are used in online retail to:

- Respond to commonly asked questions about products, shipping, and return policies.
- Help customers find products by narrowing down options based on preferences.
- Provide order status information and basic account support. And
- Lessen the workload for human customer service representatives.

Previous versions of chat-bots were largely rule-based and relied on keyword matching and pre-written scripts [?]. Due to their limited flexibility, these systems often failed to handle unexpected or complex requests.

Recent updates and the introduction of Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs) have opened up new horizons for chat-bots [?]. They are now able to use the data provided to them to respond more naturally and effectively, with relevant context.

Recent developments in Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs) have enabled chat-bots to understand questions asked in

more natural language and generate contextual answers using information from product catalogs, knowledge bases, or documentation [?].

In the context of *ShopSense*, the chat-bot is designed to:

- Answers product-related questions (e.g., “Motherboard for Intel”).
- Provides answers to order and payment-related queries.
- Helps users browse categories and apply various filters.
- Acts as a gateway to recommendation and semantic search systems.

To ensure relevant and timely answers, as well as being easy to implement, this solution is built with a combination of local or hosted Large Language Model (LLM), a database, and orchestration tools [?].

2.2.3.2 Semantic Search in Product Catalogs

Traditional e-commerce websites typically rely on keyword matching with product titles and descriptions. This approach is difficult to handle misspellings, synonyms, and more complex or conversational queries. Semantic search overcomes these limitations by encoding queries and products as vectors in a high-dimensional embedding space, where similarity is determined based on semantic relationships, not just specific word matches [1].

Modern semantic search pipelines typically encode text data using a sentence embedding model, then perform an approximate nearest-neighbor search on a vector index [1]. This makes it possible to:

- Understand the intent of a user’s query in natural language (e.g., “give me price an SSD”).
- Find relevant products even if the query words do not exactly match the product title.
- Combine vector similarity with keyword filters (price, category, brand) for hybrid search.

ShopSense uses semantic search to index product titles, descriptions, and specific attributes. When a customer submits a query, the system converts it into an embedding, searches the database for the most similar products, and displays ranked results on the front-end. This strategy is especially useful for catalogs with a high product variety and non-standard nomenclature, which is very common for small and medium-sized online retailers.

2.2.3.3 Recommendation Systems (Content-based, Collaborative)

Another important AI component in online commerce is the recommendation system, which helps customers find products that they might not find directly through search, but are likely to buy [?]. The three main types of recommendation systems are:

- **Content-based filtering:** Suggests products that are similar to the product the user has viewed or purchased based on product attributes (such as category, brand, features, and text description). This method works well with minimal user interaction data if there is sufficient product information [?].
- **Collaborative filtering:** Identifies products that “similar users” have liked by analyzing the interaction patterns of many users (e.g., ratings, purchases, views). While this approach can capture complex behavioral patterns, it typically requires large datasets and careful resolution of cold-start and sparsity issues [2].
- **Hybrid approaches:** Hybrid approaches use content-based and collaborative signals together to provide more reliable recommendations, overcoming the limitations of both approaches [?].

Due to the limited user interaction history, implementing a fully data-driven collaborative solution may not be practical for small and medium-sized e-commerce sites at the outset. For this reason, *ShopSense*’s recommendation module is designed as a hybrid system that:

- Displays “related products” and “customers also viewed” sections on the product page.

- Uses content-based similarity (via product embeddings) to generate related product suggestions [2];
- Incorporates simple collaborative signals when available (e.g., co-viewed or co-purchased products).

The system can be expanded with more sophisticated collaborative models as more interaction and transaction data are gathered over time.

2.2.4 Online Payment Integration

Processing payments securely and conveniently is an essential part of any e-commerce business. Cart abandonment rates can be greatly increased by friction and uncertainty introduced by poorly integrated payment flows[?]. Supporting locally acceptable payment methods is particularly crucial in developing nations, where digital financial literacy and trust in online payments are still developing [?].

Modern payment integration typically involves:

- Using API's to connect to third-party payment gateways.
- Securely processing customer and transaction data.
- Supporting callbacks or web-hooks for payment confirmation.
- Managing various payment stages (pending, successful, failed, refunded) are all common components of modern payment integration.

The *ShopSense* platform includes several payment methods, including local mobile banking (bKash), global card payments (Stripe), and Cash on Delivery, to offer flexibility and promote a gradual shift towards purely digital transactions [?].

2.2.4.1 Stripe and Global Card Payments

Stripe is a widely used international payment gateway that supports credit and debit card payments, recurring billing, and multiple currencies. Stripe offers the following benefits for developers:

- Well-documented REST APIs and SDKs for common languages and frameworks;

- Secure hosted checkout pages and client-side libraries to reduce PCI-DSS compliance burden;
- Webhooks for receiving asynchronous payment events (e.g., successful payment, refund).

Integrating Stripe into an e-commerce platform typically involves creating checkout sessions on the server, redirecting users to Stripe’s hosted payment pages, and handling callback events to update order status. In *ShopSense*, Stripe is used to process international or card-based payments, providing a secure and standardized way to handle customer card data without exposing sensitive information to the application server.

By enabling card payments alongside local methods, the platform addresses a key barrier identified in prior work: the need for multiple convenient and trustworthy payment options to increase online purchase adoption [?].

2.2.4.2 bKash and Mobile Payments in Bangladesh

bKash is one of the most widely used mobile financial services in Bangladesh, offering mobile wallet functionality, person-to-person transfers, and merchant payments. For many users, bKash is a more familiar and reliable digital transaction medium than traditional card payments. By integrating bKash into e-commerce platforms, user acceptance and transaction volume in the local context increase significantly [?].

A typical bKash integration for online retailers typically involves the following steps:

- Collecting an access token by authenticating with the bKash API.
- Initiating a tokenized checkout or payment request from the merchant’s server.
- Redirecting the customer to a secure bKash payment interface (or mobile app).
- Handling a callback URL or polling mechanism to confirm the transaction status.
- Handling error cases such as timeouts, canceled payments, or insufficient balance.

The official sandbox and production APIs have been used to implement the bKash integration in *ShopSense*. When the customer selects the bKash payment method during checkout, he or she is directed to complete the payment on the bKash interface through the platform's tokenized checkout flow. Once the transaction is confirmed, the system automatically updates the status of the associated order.

This design is in line with other research on mobile pay uptake, which highlights the significance of perceived security, clear feedback, and seamless integration [?]. It also helps lower the financial and logistical risks related to Cash on Delivery, which has been found to pose serious operational difficulties for e-commerce companies in developing regions [?].

2.3 Summary of Reviewed Work and Identified Gaps

This literature review shows the reality of current gaps and limitations in existing e-commerce sites, specially for small merchants in countries like ours:

- **Lack of Intelligence in Product discovery :** Almost all available platforms provides only keyword search, Which make it difficult for customers to search for complex queries. Semantic search and AI-collaborative discovery are missing in small sites or only available in large MNC's or international platforms[1].
- **Lack of personalization:** Product visibility based on users behavior is uncommon in normal sized e-commerce platforms. This also compromised the opportunity of up-selling or cross-selling which may bring negative affect on user engagement [?].
- **Slow or Delayed customer support:** Customer questions are mostly handled mainly through phone calls, messenger and whats-app , without integration into the e-commerce workflow. This leads to delays, inconsistent responses, and limited re-usability of support knowledge [?].
- **Payment integration compromise:** Local platforms are mainly known to accept COD, some local MFS (like bKash) but they lack of integration card payment system. This increases checkout friction, cart abandonment, and logistical complexity [?, ?, ?].

- **Weak data collection and analytics:** Interaction and transaction data are often not systematically captured or analyzed. Consequently, merchants lack actionable insights into customer behavior, product performance, or the effectiveness of promotions [?].

Figure 2.1 provide the top level view of these lacking and provide a small overall or understanding the struggles of small sized sites and the lack of customization which is the reasons of failing.

The *ShopSense* platform is designed to address these gaps by:

- Product discoverability is improved by integrating semantic search and AI-based recommendations [1, ?].
- Conversational assistance is provided by an intelligent chat-bot [?].
- Support Structured interaction and transaction data are captured for use in analytics and future decision support [?].
- ShopSense targets the identified gaps and existing e-commerce platforms in this conceptual overview [?].

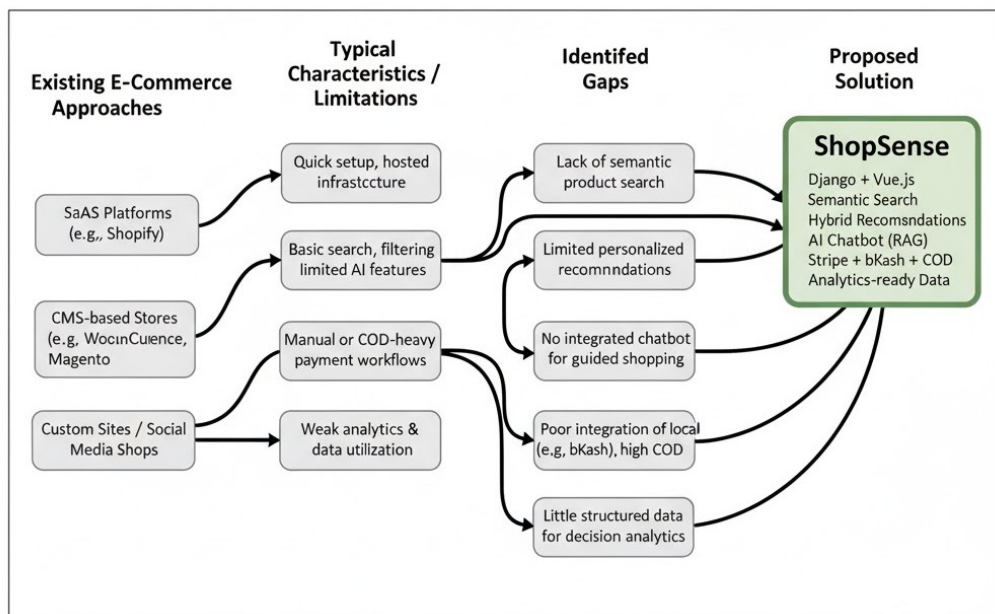


Figure 2.1: ShopSense targets the identified gaps and existing e-commerce platforms in this conceptual overview.

2.4 Conclusion

This chapter reviews existing e-commerce platforms, web frameworks, AI techniques for chatbots, semantic search, and recommendation systems, alongside online payment integrations relevant to the Bangladeshi context[?, ?, ?, ?, 1, ?, 2, ?]. The analysis highlights several shortcomings of typical small and medium-sized online stores, such as limited intelligence in product discovery, insufficient personalization, fragmented customer support, incomplete payment integration, and weak analytics (summarized in Figure 2.1).

These observations motivate the design of *ShopSense*. *ShopSense* aims to provide an integrated, AI-enhanced e-commerce platform with semantic search, recommendations, chat-bot assistance, and multi-gateway payment support. The next chapter presents the design methods and system architecture used to realize this platform.

3. Methodology and System Design

3.1 Introduction

The methods and procedures used to design the ShopSense platform, as well as the system architecture and key components, are described in this chapter. The software process model used for the project is first described, followed by the functional and non-functional requirements, the overall system architecture, and the detailed design of the backend, frontend, AI components, and payment subsystems. Lastly, it discusses the important performance and security factors that influenced the design choices.

3.2 Software Process Model and Methodology

We build Shopsense using an iterative similar method the like agile method instead of the waterfall method. The work was divided into phases in accordance with the Gantt chart in Chapter 1 (Figure 1.1), with each phase providing an incremental set of features:

- **Phase 1: Planning and Requirements-** Analysis of current e-commerce sites, literature review, and definition of functional and non-functional requirements.
- **Phase 2: Core Platform Development-** Implementation of core Django Database classes, REST APIs Endpoints, and basic static pages for products, cart, and orders.
- **Phase 3: Payments and Advanced Features-** Integration coupon system, different payment methods like MFS and Card payment and fine tuning of checkout flows.

- **Phase 4: AI/ML Components** Development of semantic search, recommendation engine, and chat-bot using Lang-Chain and ChromaDB.
- **Phase 5: Testing and Deployment-** Functional testing, performance checks on key workflows, and deployment configuration on a cloud platform.

The team evaluated progress at the conclusion of each phase, noted problems, and modified the backlog of work for following iterations. This methodical approach allowed for early validation of key functionality (such as basic cart and checkout) prior to investing in more sophisticated AI features, and also enabled design modifications depending on implementation and testing outcomes.

3.3 System Requirements

The functional and non-functional needs of the *ShopSense: Intelligent E-Commerce Platform* are described in this section. The issue statement, literature analysis, and observation of standard procedures in already-existing online stores were used to determine the needs.

3.3.1 Functional Requirements

Functional requirements outline what the system must accomplish from the viewpoints of administrators, clients, and AI services.

Customer-related Requirements

- **User registration and login:** The system must enable users to register, log in, and keep a personal profile with basic data and stored addresses.
- **Product browsing:** The system will provide products arranged by categories and sub-categories, supporting sorting (e.g., most-viewed, price, popularity) and filtering (e.g., category).
- **Product details:** The system must offer a comprehensive product view that includes stock information, available variations (such as size and color), price, description, and photos.

- **Shopping cart management:** Customers should be able to add products to their cart, change quantities, delete things, and display a summary of their cart that includes discounts and subtotals.
- **Checkout process:** Customers can enter or choose shipping addresses, select a payment option, and examine the order before confirmation using the system's multi-step checkout flow.
- **Order placement and tracking:** Following a successful checkout, the system will establish orders and enable consumers to monitor order history and current order status.
- **Coupon usage:** Customers will be able to apply valid coupon codes to qualifying orders and see the resultant discounts.
- **Payment options:** The system must accept a variety of payment methods, such as card payments via Stripe, mobile payments via bKash, and cash on delivery (COD), with clear indications of fees or restrictions where relevant.
- **AI chat-bot assistance:** After the user has been authorized, the system will offer an AI-powered chat-bot that can respond to inquiries about the product, help with navigation (such as locating goods by criteria), and provide basic order details.
- **Semantic search:** Customers can use natural language queries to search for products, and even if the query does not precisely match product titles, the system returns results that are semantically relevant.
- **Recommendations:** On pertinent sites, the system will provide tailored or context-aware product recommendations, such as "related products" and "customers also viewed."

Administrator and Staff Requirements

- **Product management:** Using a backend interface, administrators add, modify, and remove products, categories, and subcategories.
- **Inventory management:** Administrators manage stock levels and indicate whether a product is active or inactive.

- **Order management:** Administrators examine payment details, inspect orders, and adjust order status (such as pending, paid, shipped, delivered, or canceled).
- **Coupon management:** Administrators generate and oversee coupon codes, including their discount type, value, expiration date, and usage restrictions.
- **User management:** Administrators examine registered users, change roles as needed, and deal with account-related problems.
- **Analytics and reports (basic):** To aid in decision-making, the system offers fundamental indicators including the quantity of orders, total revenue, popular products, and coupon utilization.

AI and Integration Requirements

- **Product data ingestion:** Product information will be ingested into the AI as resources (vector database, recommendation engine) to get updated data from searches and recommendations.
- **Chat-bot backend selection:** Customizable chatbot backends, including local LLM (via Ollama) and a mock or external provider, will be supported and can be changed by environmental variables.
- **Payment gateway integration:** The system shall integrate with Stripe and bKash APIs for payment initialization, status verification, and web-hook handling.
- **API access:** The platform shall expose REST APIs for cart operations, product retrieval, recommendations, coupon validation, and chat-bot communication to support the Vue.js front-end and possible future clients.

3.3.2 Non-Functional Requirements (Performance, Security, Usability)

Non-functional requirements describe constraints and quality attributes that the system should satisfy.

Performance Requirements

- The system shall respond to typical page requests (home page, catalog, product detail) within an acceptable time under normal load (e.g., target < 1.5 seconds server response time on a mid-range hosting environment).
- Semantic search and recommendation queries shall be optimized to return results within a practical latency budget (e.g., target < 1 second additional delay for search results).
- The platform shall efficiently handle concurrent users typical of a small to medium-sized store; heavy load or large-scale optimization is considered future work.

Security Requirements

- All authenticated pages and APIs shall enforce secure session management and authorization checks to prevent unauthorized access to user or administrative data.
- The system shall protect against common web vulnerabilities such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF) using Django's built-in security mechanisms and secure coding practices.
- Sensitive configuration values, such as secret keys and API credentials for Stripe and bKash, shall be stored in environment variables and not hardcoded in the source code.
- Payment card data shall never be stored on the application server; Stripe's client-side or hosted checkout solutions shall be used so that payment data is processed directly by the gateway.

Usability and Reliability Requirements

- A good website is one that has seamless optimization for all devices, device dependency should not be a barrier to user experience, and design consistency is a must for user comfort.

- Proper error description and details make users more comfortable while facing any error in the web app. Proper guidance or hints can be useful to make users navigate more willingly.
- The system should log important events (e.g., payment callbacks, failed payments, order status changes) to support debugging and operational monitoring.
- The platform should provide proper error message when AI-features are unavailable (e.g., fallback to keyword search or mock chat-bot responses).

3.4 System Design and Architecture

This chapter described the methodology and system design of the ShopSense platform. An iterative, Agile-inspired process was adopted to develop the system in phases, guided by clearly defined functional and non-functional requirements.

3.4.1 High-Level Architecture Diagram (Django + Vue + APIs + DB)

A modular, service-oriented design is followed by the ShopSense architecture (Figure 3.1). The Django backend is communicated with by the Vue.js frontend through RESTful APIs. The relational database is interacted with by the backend for transactional data, and AI components are utilized for search, recommendations, and chatbot responses. Secure HTTP APIs and webhooks integrate payment gateways (Stripe and bKash).

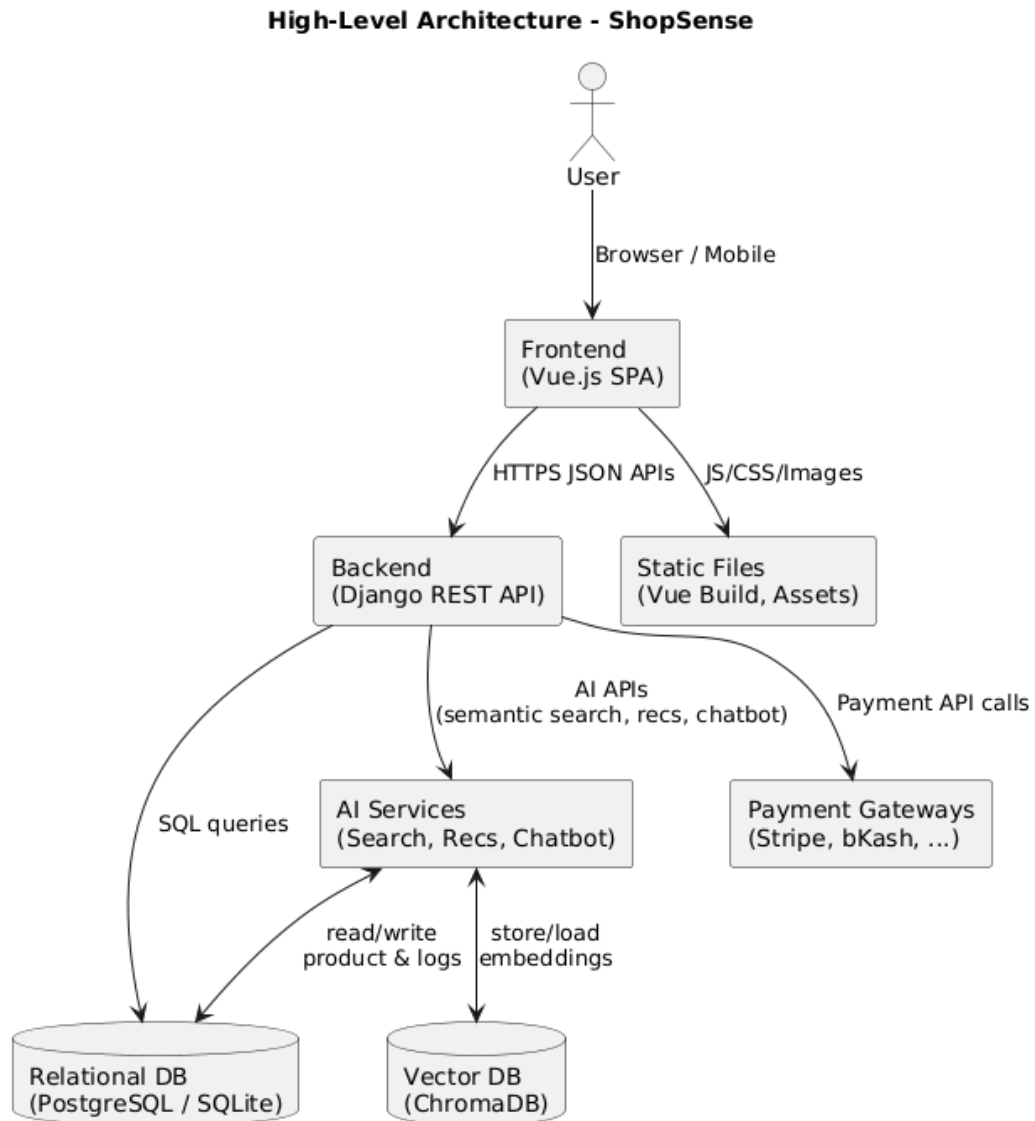


Figure 3.1: High-level architecture of the ShopSense platform, in which interactions among the Vue.js front end, Django backend, relational database, AI services, and payment gateways are visualized.

3.4.2 Data Flow Between Front-end, Backend, and AI Services

The primary data flows in the system are illustrated in Figure 3.2. HTTP requests from the Vue.js front-end to the Django REST API are initiated by typical user actions—such as searching for products, viewing recommendations, or interacting with the chat-bot. Coordination with the relational database and AI services is then

handled by the backend as needed.

- **Product browsing and search:** The frontend sends search queries and filter parameters to the backend. For semantic search, the backend encodes the query into an embedding, queries the vector database, and returns ranked products to the frontend.
- **Chatbot interaction:** The frontend sends user messages to a chatbot API endpoint. The backend constructs a context, retrieves relevant product or documentation snippets via ChromaDB, forwards the prompt to the configured LLM backend (Ollama or external), and returns a generated response.
- **Recommendations:** When a user opens a product page or the cart, the frontend requests recommendations. The backend computes or retrieves recommended products using a hybrid strategy and returns them to be displayed alongside the primary content.

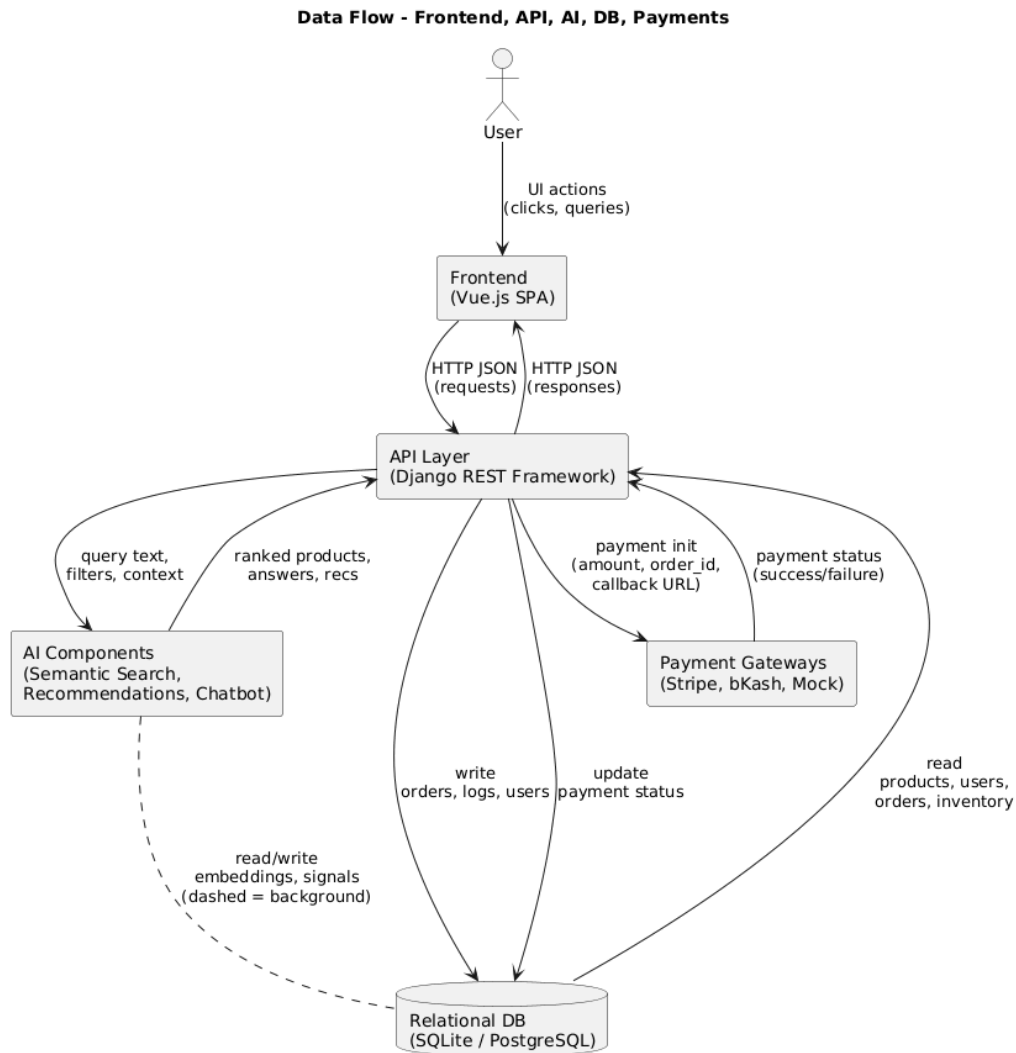


Figure 3.2: Data flow between the Vue.js frontend, Django REST API, AI components (semantic search, recommendations, chatbot), and the relational database.

3.5 Backend Design (Django)

The backend is implemented using Django and Django REST Framework. It is organized into multiple Django apps corresponding to core business domains, such as products, orders, cart, coupons, search, recommendations, and chatbot.

3.5.1 Database Schema and Models (Products, Users, Orders, Cart, Coupons)

The database schema follows a relational model using Django's ORM. The key entities are illustrated in Figure 3.3.

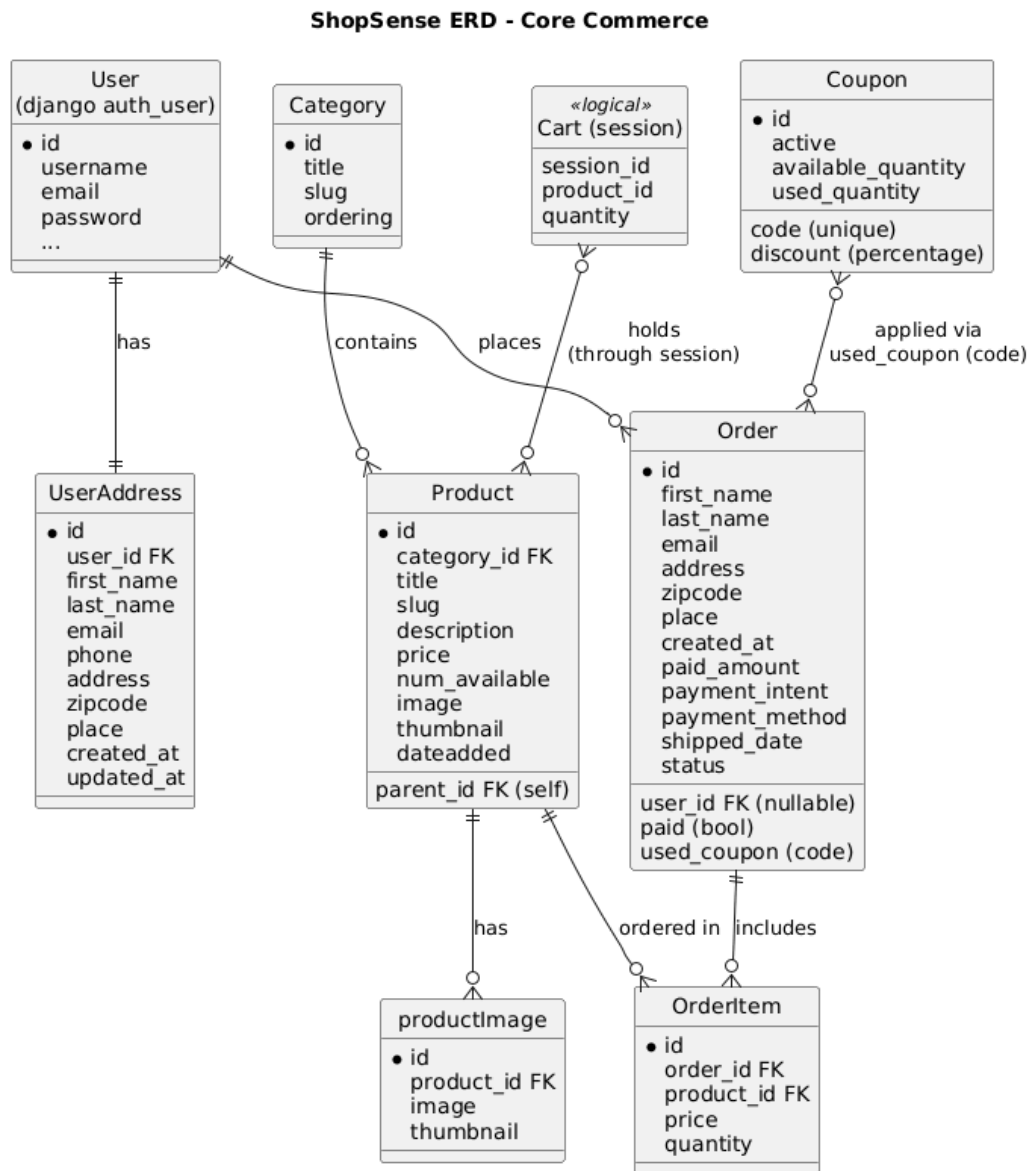


Figure 3.3: Entity–relationship diagram of the ShopSense database schema, including users, products, categories, carts, orders, order items, and coupons.

Core Entities

- **User:** Extends Django's built-in user model (or uses a custom user model) to store customer information such as name, email, and addresses.
- **Category:** Represents high-level product categories and optional subcategories for organizing the catalog.
- **Product:** Stores product attributes including title, description, price, stock quantity, category, image paths, and optional variant information.
- **Cart and CartItem:** Represent a user's active shopping cart and its items, with references to products and quantities. Sessions are used for anonymous carts when users are not logged in.
- **Order and OrderItem:** Capture confirmed purchases, linking users, products, quantities, applied discounts, payment status, and shipping information.
- **Coupon:** Represents discount codes with fields such as code, discount type (percentage or fixed amount), value, expiration date, minimum order amount, and usage limits.
- **Payment:** Stores payment-related metadata (transaction IDs, gateway type, status) received from Stripe, bKash, or the mock gateway.

3.5.2 REST API Endpoints (Cart API, Product API, Recommendations API, Coupon API)

The Django REST Framework is used to implement a set of JSON APIs consumed by the Vue.js frontend. Table 3.1 summarizes the main endpoints.

Table 3.1: Overview of key REST API endpoints in the ShopSense backend.

Endpoint	Method(s)	Description
/api/products/	GET	List products with pagination, filters, and sorting.
/api/products/{id}/	GET	Retrieve details of a single product.
/api/search/semantic/	POST	Perform semantic search over the product catalog.
/api/cart/	GET, POST	Retrieve current cart; add items to cart.
/api/cart/update/	POST	Update quantities of items in the cart.
/api/cart/remove/	POST	Remove items from the cart.
/api/cart_count/	GET	Return the number of items in the cart.
/api/recommendations/{product_id}/	GET	Retrieve recommended products related to a given product.
/api/coupons/validate/	POST	Validate coupon code and return discount information.
/api/checkout/initiate/	POST	Initiate checkout and start payment flow.
/api/chatbot/message/	POST	Send user message to chatbot and receive response.

Additional endpoints are provided for user registration, authentication, order retrieval, and administrative operations as required.

3.6 Frontend Design (Vue.js)

The frontend is implemented as a single-page application using Vue.js. It consumes the Django REST API and provides a responsive, interactive user interface.

3.6.1 Component Structure (Home, Catalog, Product Detail, Cart, Checkout, Profile)

The main view components are organized according to major user tasks, as depicted in Figure 3.4:

- **HomeView:** Landing page showcasing featured products, categories, promotional banners, and entry points to search and recommendations.
- **CatalogView:** Displays products by category with filtering, sorting, and pagination controls.
- **ProductDetailView:** Shows detailed information for a single product, including gallery, variants, reviews (if implemented), and a section for related products.
- **CartView:** Displays current cart items, quantities, subtotal, applied coupons, and options to update or remove items.
- **CheckoutView:** Implements the stepwise checkout process (address, payment method selection, order review).
- **ProfileView:** Allows users to view and update profile information, manage saved addresses, and see order history.
- **ChatbotWidget:** Provides a floating chat interface that can be opened from any page to interact with the AI assistant.

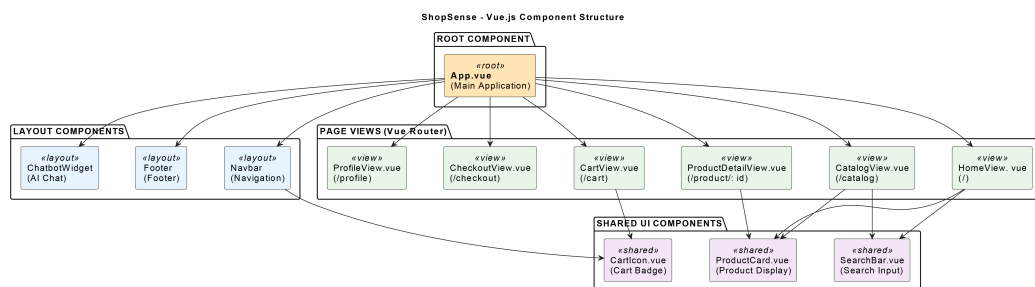


Figure 3.4: Simplified Vue.js component structure for the ShopSense frontend.

3.6.2 State Management (Vuex) and Routing

Application-level state such as the shopping cart, authenticated user information, and some UI preferences is managed using Vuex (or the equivalent state management library used in the implementation). Separate Vuex modules are used for:

- **auth:** Handles authentication tokens, login/logout actions, and user profile data.
- **cart:** Stores cart items, counts, and computed totals; synchronizes with back-end cart APIs.
- **products:** Caches product lists and product details to reduce redundant API calls.
- **chatbot:** Maintains current conversation state and last responses from the chatbot API.

Routing is managed using the Vue router, mapping URL paths to view components (e.g., /, /catalog, /product/:id, /cart, /checkout, /profile). Navigation guards are used to ensure that only authenticated users can access protected routes such as the profile and order history pages.

3.7 AI Components

The AI layer of ShopSense consists of three main components: the chatbot, the semantic search system, and the recommendation engine. These components share some underlying infrastructure, such as embedding models and the vector database.

3.7.1 Chatbot Architecture (LangChain + Ollama/OpenAI + ChromaDB)

The chatbot is implemented as a retrieval-augmented generation system (Figure 3.5). Key elements include:

- **Frontend chat widget:** Captures user messages and displays responses.

- **Chatbot API endpoint:** Receives messages from the frontend, attaches user/session metadata, and orchestrates the conversation logic.
- **Retrieval layer:** Uses ChromaDB to retrieve relevant product descriptions, FAQs, and documentation sections based on an embedding of the user query.
- **LLM backend:** Uses either a local LLM via Ollama or a remote LLM provider (e.g., OpenAI), selected through configuration.
- **Response generation:** Combines retrieved context documents with the user query in a prompt template and generates a response, which is then returned to the frontend.

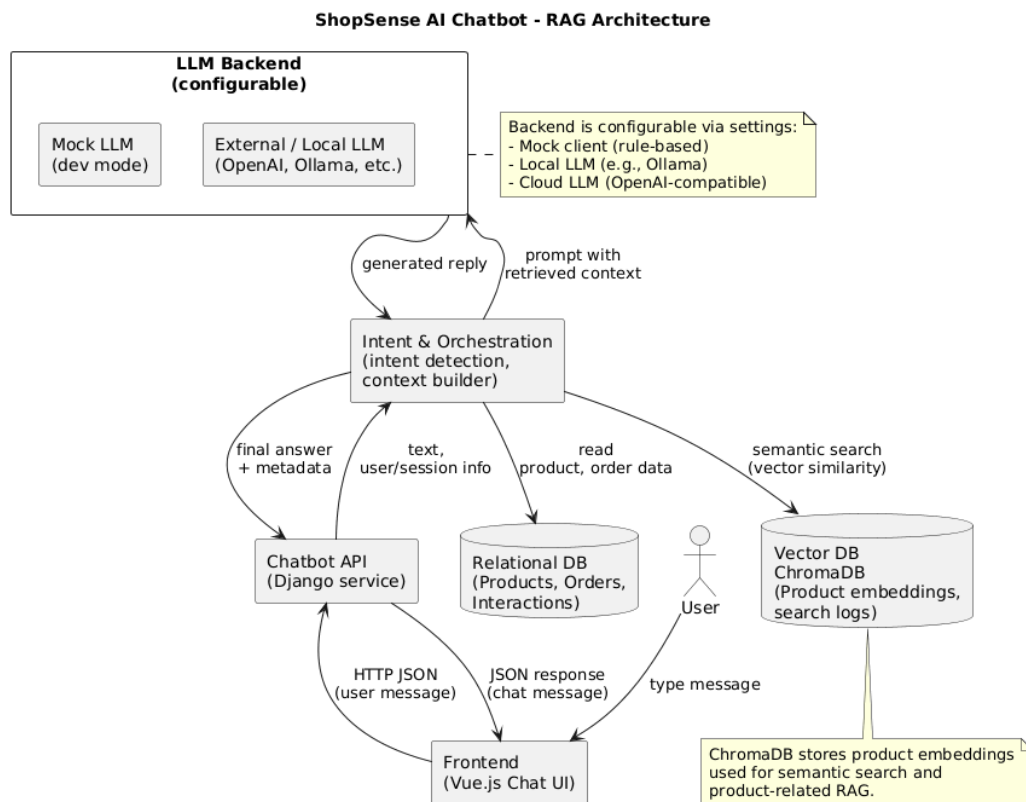


Figure 3.5: Architecture of the ShopSense AI chatbot using retrieval-augmented generation with ChromaDB and configurable LLM backends.

3.7.2 Semantic Search System (Sentence Transformers, Vector Index)

The semantic search system encodes product text (titles, descriptions, key attributes) into embeddings using sentence transformer models. These embeddings are stored in a vector database (ChromaDB) and used for nearest-neighbor search.

- **Indexing pipeline:** A backend process iterates over products in the relational database, generates embeddings, and upserts them into the ChromaDB collection with product IDs as references.
- **Query pipeline:** For each search request, the query text is encoded into an embedding, which is used to query ChromaDB for the most similar products. Results are combined with optional filters (e.g., category, price range) from the relational database.

Figure 3.6 summarizes the semantic search architecture.

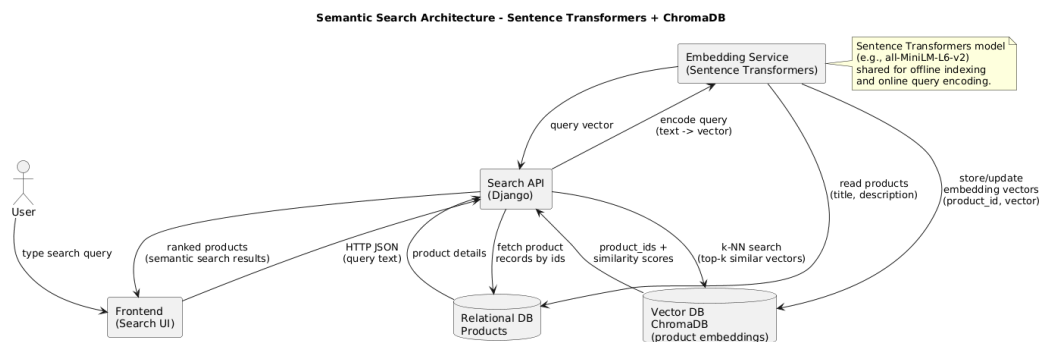


Figure 3.6: Semantic search architecture using sentence transformers and ChromaDB for vector similarity retrieval.

3.7.3 Recommendation Engine

The recommendation engine is designed as a hybrid system combining content-based and simple collaborative signals.

Content-Based Recommendations

Content-based recommendations rely on product similarity computed from text embeddings and structured attributes (e.g., category, brand). When a user views a product, the system retrieves:

- products in the same or related categories;
- products with high cosine similarity in the embedding space;
- optionally, products matching known user preferences (e.g., price range).

Collaborative Filtering

Collaborative filtering methods use historical interaction data (views, cart additions, purchases) to infer relationships between products and users. In *ShopSense*, lightweight collaborative signals are used initially, such as:

- “users who viewed this also viewed” based on co-viewing patterns;
- “frequently bought together” based on co-purchase counts when available.

These signals can be extended in future work with more advanced matrix factorization or neural models once sufficient data volume is available.

Hybrid Strategy

The hybrid strategy combines content-based and collaborative scores to rank recommendations. A weighted linear combination is used in the initial implementation:

$$\text{score}(p) = \alpha \cdot \text{sim}_{\text{content}}(p, p_{\text{ref}}) + (1 - \alpha) \cdot \text{score}_{\text{collab}}(p) \quad (3.1)$$

where:

p is the candidate product being evaluated for recommendation

p_{ref} is the reference product (the product currently being viewed by the user)

$\text{sim}_{\text{content}}(p, p_{\text{ref}})$ is the cosine similarity between the embedding vectors of product p and reference product p_{ref} , normalized to the range $[0, 1]$

$\text{score}_{\text{collab}}(p)$ is the collaborative filtering score based on user interaction patterns (co-views, co-purchases), also normalized to $[0, 1]$

α is a tunable weight parameter where $\alpha \in [0, 1]$. The default value is $\alpha = 0.7$, which emphasizes content-based similarity

Adaptive Weighting Strategy:

- In cold-start scenarios (new products or insufficient collaborative data), the system sets $\alpha = 1.0$ to rely entirely on content-based similarity
- As interaction data accumulates over time, α can be reduced (e.g., to 0.6 or 0.5) to give more weight to behavioral signals
- This adaptive approach ensures sensible recommendations even with limited data while improving personalization as the system matures

Through this approach:

- sensible recommendations are provided even in cold-start scenarios (driven by content-based similarity);
- behavioral signals are gradually incorporated as data accumulates.

3.8 Payment System Design

Stripe, bKash, and a mock gateway are integrated by the payment subsystem to support the requirements identified in the problem statement. Secure handling of payment data and clear separation between order creation and payment confirmation are emphasized by the design.

3.8.1 Stripe Integration Flow

The Stripe payments were integrated for getting payment action done. The basic flow is illustrated in Figure 3.7:

- i. The customer selects card payment during checkout and submits the order details to the backend.

- ii. The backend creates a payment intent or checkout session with Stripe, including amount, currency, and metadata (e.g., order ID).
- iii. The frontend redirects the customer to Stripe's secure payment page or uses Stripe's client-side components to collect card details.
- iv. Upon completion, Stripe sends a webhook to the backend indicating success or failure.
- v. The backend verifies the event, updates the order and payment status, and notifies the frontend.

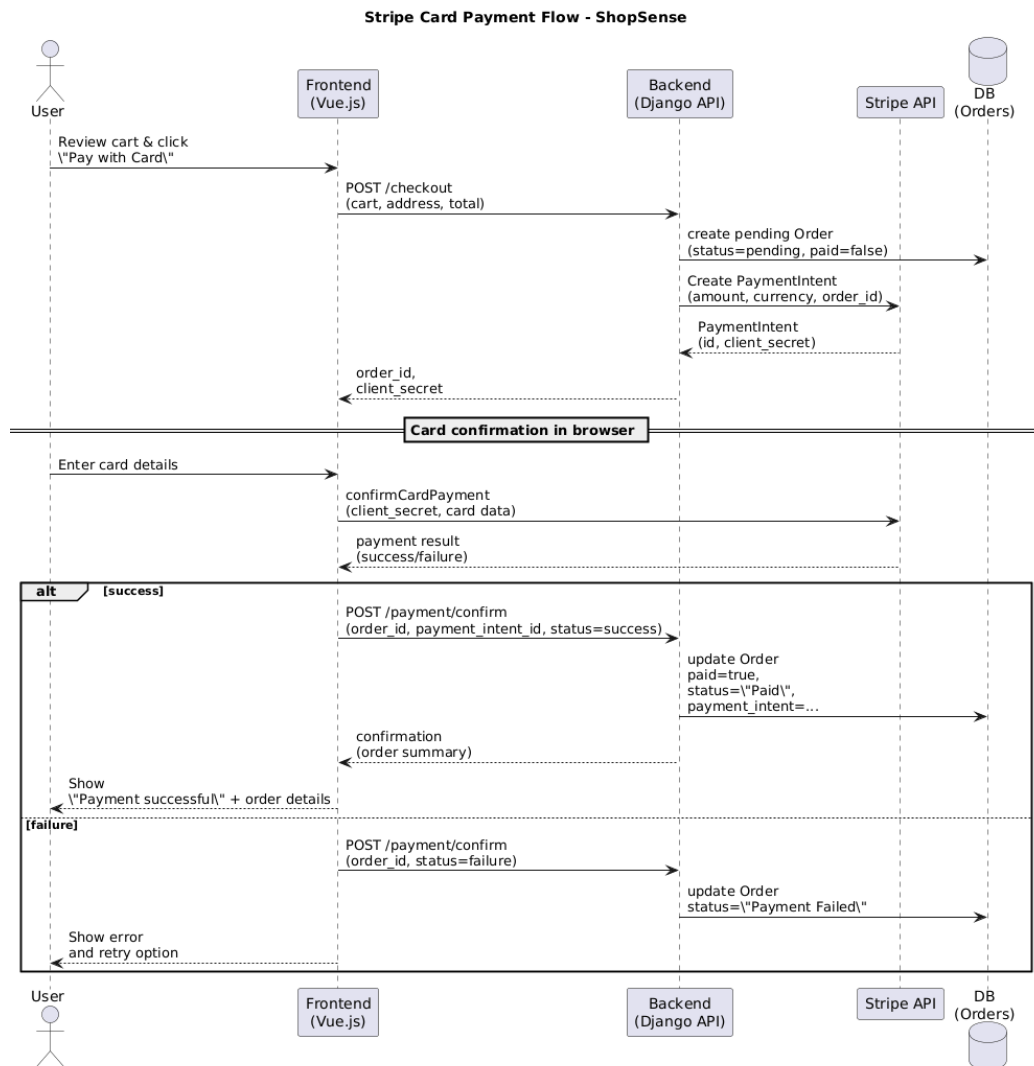


Figure 3.7: Stripe payment integration flow for card payments in ShopSense.

3.8.2 bKash Integration Flow (Tokenized Checkout)

The bKash integration follows the official tokenized checkout approach suitable for Bangladeshi merchants (Figure 3.8):

- i. The customer chooses bKash as the payment method and confirms the order on the checkout page.
- ii. The backend authenticates with the bKash API to obtain an access token if one is not already available.

- iii. The backend initiates a payment request with bKash, which returns a payment URL or token.
- iv. The frontend redirects the customer to the bKash payment interface (web or app), where the customer authorizes the payment.
- v. bKash redirects back to the merchant callback URL or sends a notification with the payment result.
- vi. The backend verifies the transaction status via the bKash API, updates the order and payment records, and informs the user.

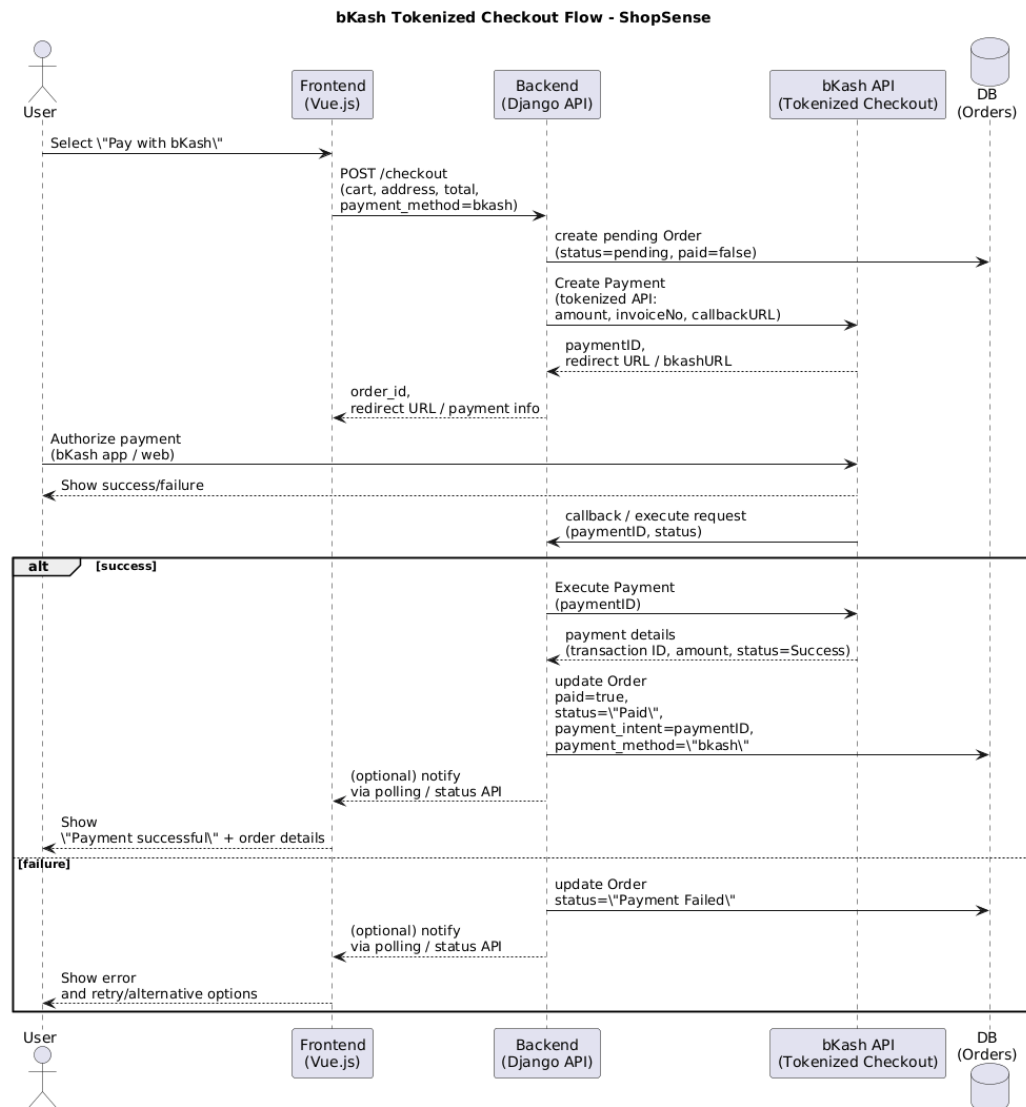


Figure 3.8: bKash checkout flow in ShopSense.

3.8.3 Mock Payment Gateway for Testing

For testing purpose at the start I build a static gateway mimicking bKash sandbox to support development and demonstrations without relying on real payment providers. The mock gateway:

- Provides some simple endpoints that mimicking the whole process for authorization, success, and failure situations;

- Instant approval or configurable delays/errors are allowed for testing.
- Structured responses similar to real gateways are returned so that the rest of the payment logic can be tested consistently.

Figure 3.9 provides an overview of the mock gateway flow.

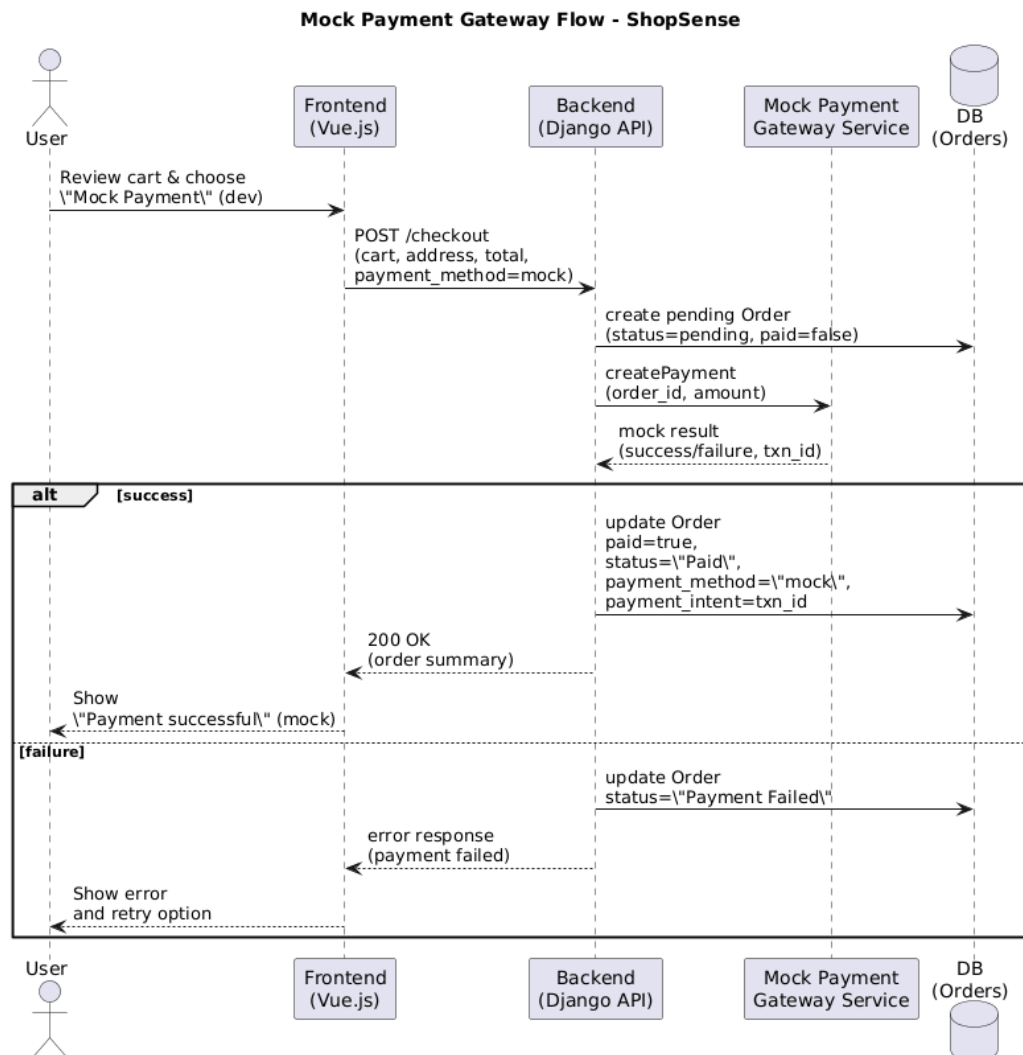


Figure 3.9: Mock payment gateway flow used for development and testing.

3.9 Security and Performance Considerations

This section outlines how the system addresses security and performance concerns at a design level.

3.9.1 Authentication, Authorization, CSRF

- **Authentication:** Django’s authentication framework is used to manage user logins, password hashing, and session management. For API calls, token-based or session-based authentication is used depending on the deployment configuration.
- **Authorization:** Access to administrative views and APIs is restricted by user roles and permissions. Regular users can only access their own profile and order data, while staff users can access management interfaces.
- **CSRF protection:** Django’s built-in CSRF middleware is enabled for state-changing operations. The Vue.js frontend includes CSRF tokens in relevant requests to prevent cross-site request forgery attacks.
- **Input validation and sanitization:** All external inputs (forms, API requests, webhook payloads) are validated on the server side. Where user-generated content is displayed, appropriate escaping and sanitization are applied.

3.9.2 Static File Optimization, Caching, Database Tuning

- **Static file optimization:** Static assets (CSS, JavaScript, images) are collected via Django’s staticfiles framework and served through an optimized mechanism such as WhiteNoise or a CDN. Minification and bundling are applied on the frontend build pipeline to reduce payload size.
- **Caching:** Frequently accessed data (e.g., home page product lists, category menus) can be cached at the view or template level. API responses that do not change frequently may also be cached for short durations.
- **Database tuning:** We created appropriate indexes on the commonly queried fields things like product category, slug, and foreign keys. We also reviewed

the query patterns to avoid any unnecessary joins or N+1 queries, especially when you're in the catalog and order-related views.

- **AI performance:** We optimized embedding generation and vector search by precomputing product embeddings and batching operations whenever possible. And in production deployments? And in production deployments, we can actually move those embedding and retrieval services to separate worker processes or servers.

So, ShopSense is ensured to remain secure, responsive, and maintainable by the combination of all the above measures. At the same time, advanced AI features and multi-payment integration are provided, making it perfect for those small to medium-scale deployments.

3.10 Comparative Analysis with Existing Platforms

Table 3.2 compares ShopSense with traditional e-commerce platforms and popular SaaS solutions, highlighting the unique value proposition and design choices of this work.

Table 3.2: Comparison of ShopSense with existing e-commerce platforms.

Feature	Traditional BD Platforms	SaaS (Shopify)	ShopSense
AI-Powered Semantic Search	Not available	Limited (via paid plugins)	Yes (built-in, ChromaDB + Sentence Transformers)
Recommendation Engine	Manual or none	Limited (basic tier) / Advanced (paid)	Yes (hybrid content + collaborative)
AI Chatbot (RAG-based)	External (Facebook Messenger, WhatsApp)	Third-party integrations only	Yes (integrated LangChain RAG)
Local Payment Integration (bKash)	Manual or semi-automated	Via third-party plugins	Native tokenized API integration
Card Payments (Stripe)	Rare or limited	Yes (standard)	Yes (full webhook support)
Cash on Delivery (COD)	Yes	Via apps	Yes (built-in)
Deployment Flexibility	Low (shared hosting)	None (SaaS only)	High (self-hosted, cloud-ready)
Customization	Limited by platform	Medium (theme + plugins)	High (full source code access)
Data Ownership	Full	Limited (vendor lock-in)	Full (self-hosted database)
Initial Setup Cost	Medium-High	Low (subscription starts \$29/mo)	Low (open-source, hosting cost only)
Scalability	Varies (often limited)	High (managed infrastructure)	Medium (configurable, horizontal scaling possible)
Analytics & Reporting	Basic or none	Advanced (paid tiers)	Custom (extendible, raw data access)

Key Observations:

- ShopSense addresses critical gaps in traditional Bangladeshi e-commerce platforms: intelligent product discovery, automated payment workflows, and conversational AI support
- Compared to SaaS solutions like Shopify, ShopSense offers greater control over data, lower long-term costs, and deeper customization capabilities suitable for SMEs with technical capacity

- The trade-off is that ShopSense requires self-hosting and maintenance, whereas SaaS platforms provide managed infrastructure
- For research and educational purposes, ShopSense demonstrates how open-source tools can achieve enterprise-grade features at a fraction of the cost

3.11 Engineering Considerations

This subsection discusses the engineering principles and design trade-offs that guided the development of ShopSense.

Scalability Considerations

Several architectural decisions were made to support future scalability:

- **Database indexing:** Indexes were created on frequently queried fields such as product slug, category, foreign keys (product_id, user_id), and dateadded. These indexes significantly reduce query latency for catalog browsing and order retrieval operations.
- **Caching strategy:** Django's caching framework is configured to support Redis or Memcached backends for:
 - Session data (reducing database load)
 - Frequently accessed views (e.g., homepage, category pages)
 - API responses (e.g., cart count, product lists)

In the current prototype, file-based caching is used for simplicity, but production deployments can enable Redis with minimal configuration changes.

- **AI service separation:** Semantic search, recommendation computation, and chatbot inference are designed as loosely coupled modules. These can be deployed as:
 - Separate Django management commands or Celery tasks for offline processing
 - Independent microservices accessed via REST or gRPC APIs

- Containerized services orchestrated with Docker Compose or Kubernetes
- **Horizontal scaling:** The Django backend is stateless (session data stored externally), allowing multiple application instances to be deployed behind a load balancer (e.g., Nginx, HAProxy, or cloud load balancers).
- **Static file delivery:** WhiteNoise serves static files efficiently in production. For high-traffic scenarios, static assets can be offloaded to a CDN (e.g., Cloudflare, AWS CloudFront).

Maintainability Considerations

The codebase is structured to promote long-term maintainability:

- **Modular app structure:** Django apps are organized by domain (store, cart, order, coupon, search, recommendations, chatbot, userprofile), promoting separation of concerns and making it easier to locate and modify functionality.
- **Configuration management:** Sensitive credentials and environment-specific settings are externalized using environment variables loaded via `python-dotenv`. This prevents hardcoding secrets and simplifies deployment across development, staging, and production environments.
- **Documentation:** Inline code comments, docstrings, and this thesis document provide comprehensive system documentation. API endpoints are self-documenting via Django REST Framework's browsable API interface.
- **Version control and CI/CD readiness:** The project is maintained in a Git repository with a clear branch strategy. The modular structure and automated tests (discussed in Chapter 4) facilitate continuous integration pipelines.

Extensibility Considerations

The architecture is designed to accommodate future enhancements:

- **RESTful API design:** Django REST Framework provides a stable, versioned API contract. Future clients (mobile apps, third-party integrations) can consume these APIs without requiring backend changes.

- **Payment gateway abstraction:** Payment methods are implemented with a common interface pattern. Adding new gateways (e.g., Nagad, Rocket) requires implementing the same interface without modifying existing checkout logic.
- **AI model swapping:** The chatbot supports configurable LLM backends (Ollama, OpenAI, mock) through environment variables and adapter classes. Similarly, embedding models can be swapped by changing the SentenceTransformer model name in configuration.
- **Plugin-ready architecture:** Future features such as product reviews, wishlists, or loyalty programs can be added as new Django apps without disrupting existing modules.

Technology Stack Justification

The choice of technologies was driven by the following considerations:

- **Django (Python):** Selected for rapid development, built-in ORM and admin interface, strong security features, and compatibility with Python's rich AI/ML ecosystem (PyTorch, Transformers, scikit-learn, LangChain).
- **Hybrid Vue.js frontend:** Embeds Vue 3 into Django templates rather than building a separate SPA. This approach:
 - Eliminates the need for a complex frontend build pipeline
 - Preserves server-side rendering benefits (SEO, faster initial page loads)
 - Still provides modern reactive UI components (search bar, chatbot widget, cart badge)
- **PostgreSQL:** Chosen for its reliability, ACID compliance, advanced indexing, JSON field support, and excellent Django integration. SQLite is used in development for simplicity.
- **ChromaDB:** A lightweight, embedded vector database suitable for prototype and small-to-medium scale deployments. For production scale, migration paths exist to Pinecone, Weaviate, or Qdrant.

- **Sentence Transformers:** Provides state-of-the-art sentence embeddings with reasonable computational requirements. Models like all-MiniLM-L6-v2 balance accuracy and inference speed.

Design Trade-offs

Several trade-offs were made during the design process:

- **Hybrid frontend vs full SPA:** Sacrifices some client-side sophistication and component reusability for simpler deployment, better SEO, and lower infrastructure complexity. This is appropriate for small-to-medium merchants but may need revisiting for highly interactive or mobile-first applications.
- **Session-based cart:** Simpler to implement and suitable for the current scale, but does not support persistent carts across devices or long-term cart recovery. Migration to a database-backed cart model is straightforward if needed.
- **Lightweight AI models:** Prioritizes feasibility, response time, and resource efficiency over maximum accuracy. For example, all-MiniLM-L6-v2 embeddings are sufficient for semantic search in a catalog of thousands of products, though larger models could improve performance with more computational resources.
- **Single-vendor design:** The current data model assumes a single merchant store. Supporting multi-vendor marketplaces would require significant extensions (vendor models, commission tracking, split payments) but the modular architecture provides a foundation for such growth.
- **Mock payment gateway:** Enables testing and demonstration without live credentials, but production deployments must replace mock flows with real gateway integrations and undergo security audits.

These engineering considerations demonstrate that ShopSense is not only functional but also architected with scalability, maintainability, and extensibility in mind, making it a solid foundation for real-world deployment and future research.

3.12 Conclusion

So, all the methodological approach and full system design of the *ShopSense* webapp are discussed in this chapter. We actually developed the system step-by-step, using an iterative and Agile-inspired process, all based on clearly defined functional and non-functional requirements.

AI, front-end and backend design, overall architecture , user flow of the payment systems, and important security and performance issues? They are all discussed right here. And as for the next chapter? The process of implementing these designs and the technologies used to implement the platform are going to be detailed.

4. Implementation

4.1 Introduction

The development environment and technological stack, key e-commerce capabilities, AI components, payment integrations, deployment configuration, and a visual overview of the built user interface via screenshots are all covered in this chapter's practical implementation of the *ShopSense* platform. While architecture and design were the main topics of Chapter 3, this chapter detailed how those designs were implemented in code and appeared in the finished product.

4.2 Development Environment and Tech Stack

4.2.1 Backend Tools (Django, Python Libraries)

The main part backend is implemented using **Django**. Django provides the ORM, authentication, admin interface, request handling, cart management, checkout, semantic search, recommendations, and chatbot communication. Django provides core backend capabilities such as the Object-Relational Mapper (ORM), authentication system, admin interface, session management, and middleware-based request handling. The backend is rendered using django views. Where the frontend gets data using context-variable which are rendered by the Vue.js component. The application is built with:

- **Python 3.11** as the main programming language as Django is an Python framework.
- **Django 5.x:** is used for URL routing, models, views, authentication, session middleware, and admin interface.

- **SQLite:** as the development database and PostgreSQL as the production database (via psycopg2-binary), configured through Django's settings module. And **PostgreSQL** in production (via psycopg2-binary) configured through the Django settings module.
- **Gunicorn:** WSGI HTTP server used as the application server in production.
- **WhiteNoise:** Django's request/response processing Middleware used to serve static files in production without requiring a separate web server.
- Supporting libraries such as python-dotenv for secret data like API keys etc. , Pillow for image modifications (thumbnails), and HTTP clients (requests/ httpx) for third party redirection.

The project layout under the aistore directory follows the standard Django pattern with separate apps for core domains:

- **core:** generic files like frontpage, category detail, and other general views and templates.
- **store:** for store related product and category table and views.
- **cart:** for passing cart data to the all pages and checkout flow.
- **order:** to control all order and order item table and related views.
- **coupon:** to creation and modification coupon related table and validation logic.
- **search:** to implement semantic search logic and embedding management commands.
- **recommendations:** to write recommendation algorithms and related utilities.
- **chatbot:** to handle chatbot related logic and API endpoint.
- **userprofile:** to handle user profile, addresses, and signup logic and tables.

Each app encapsulates its own models(DB Table), views(templates), and business logic, following Django's recommended modular structure.

4.2.2 Frontend Tools (Vue 3, Vuex, TailwindCSS)

Nomally vue apps are single page app but here we do different like **hybrid architecture**, the frontend employs a hybrid design in which Vue 3 is incorporated into Django templates. This method combines reactive client-side interaction with standard server-side rendering.

Key technologies include:

- **Vue 3:** One of the key technologies is Vue3, a progressive JavaScript framework that is loaded through a CDN (<https://unpkg.com/vue@3/dist/vue.global.js>) and utilized to improve interactive areas in Django templates, such the search bar, navigation bar, and chatbot widget in `base.html`.
- **Vuex 4:** This centralized state management library, which is also loaded via CDN (<https://unpkg.com/vuex@4.0.2/dist/vuex.global.js>), is used to keep track of shared state between Vue component instances mounted in the layout, particularly the global cart item count `numItems`.
- **Django Templates:** Django's template engine, with `base.html` as the master layout, is used for server-side rendering. Core routes such as `/`, `/<category_slug>/`, `/<category_slug>/<product_slug>/`, `/cart/`, `/cart/checkout/`, `/myaccount/`, `/login/`, and `/signup/` are defined in Django's URL configuration and mapped to view functions that render HTML templates.
- **Tailwind CSS:** Utility-first CSS framework included via CDN (<https://cdn.tailwindcss.com>) and configured in `base.html` to provide a responsive, dark-mode-enabled UI without extensive custom CSS files.
- **Fetch API:** The native browser `fetch()` function is used for asynchronous HTTP requests to Django API endpoints such as `/api/cart_count/`, `/api/add_to_cart/`, `/api/remove_from_cart/`, `/search/api/suggestions/`, and `/api/chatbot/ask/`, enabling dynamic content updates without full page reloads.

The simplified component hierarchy used by the application is shown in Figure 3.4 in Chapter 3. This hybrid design avoids a separate frontend build pipeline and keeps deployment simple, while still providing a modern, reactive user experience.

4.2.3 AI/ML Libraries (LangChain, ChromaDB, Sentence Transformers, scikit-learn)

AI functionality in ShopSense is implemented using a combination of specialized libraries:

- **LangChain:** Orchestration framework for large language model (LLM) calls, prompt templating, retrieval-augmented generation (RAG), and integration with vector stores for the chatbot system.
- **ChromaDB:** Lightweight vector database used to store and query product embeddings for semantic search and content-based recommendations.
- **Sentence Transformers:** Pre-trained transformer models (e.g. all-MiniLM-L6-v2) to convert product descriptions and user queries into dense vector embeddings for semantic similarity matching.
- **Transformers and PyTorch:** Underlying libraries providing transformer model architectures and tensor operations.
- **scikit-learn:** Machine learning utilities including TF-IDF vectorization, cosine similarity computation, and simple dimensionality reduction for the recommendation system.
- **NumPy, Pandas, and SciPy:** Numerical computing and data manipulation libraries used to support recommendation algorithms and matrix operations.

These components are integrated into the Django backend within the `apps.search`, `apps.recommendations`, and `apps.chatbot` modules, with management commands that handle offline embedding generation and index updates.

4.3 User Interface Screenshots

This section presents representative screenshots of the implemented ShopSense platform, illustrating how the backend functionality and AI features are exposed through the user interface. All screenshots were captured from the running system in a development environment using the project's screenshots/ assets.

4.3.1 Customer-Facing Views

Homepage and Catalog

Figure 4.1 shows the main landing page of ShopSense. It includes a prominent navigation bar, hero section, and featured products drawn from the catalog. Category links allow users to quickly navigate to specific product groups.

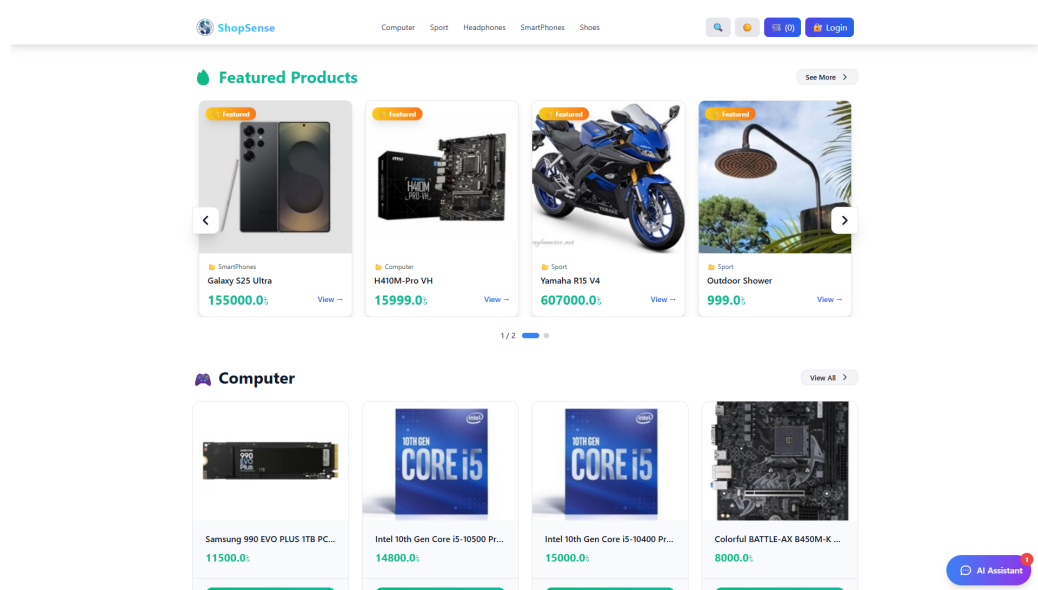


Figure 4.1: ShopSense homepage with navigation bar, search entry point, and featured products.

The catalog view in Figure 4.2 displays products within a selected category, along with filtering and sorting controls. Filters map directly to backend query parameters (e.g. price range, category, sort order), which are processed by the Django views and REST APIs described in Chapter 3.

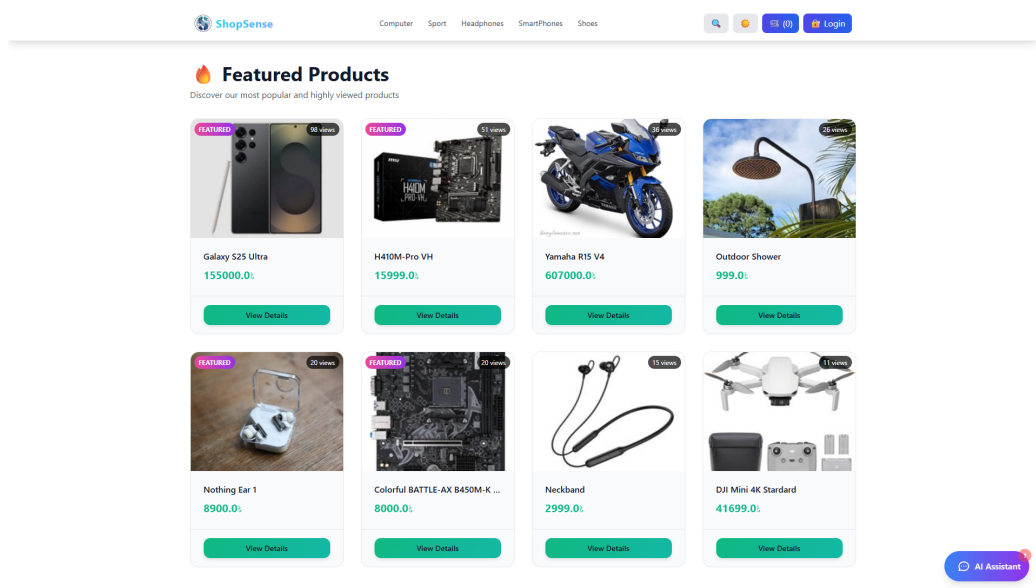


Figure 4.2: Catalog page showing product cards with filtering and sorting options.

Product Detail, Cart, and Checkout

Figure 4.3 presents the product detail page, including gallery images, price, stock information, and related products. The “Add to Cart” button triggers a call to the cart API endpoint (`/api/add_to_cart/`) via `fetch()`, which updates both the server-side session cart and the Vuex `numItems` state.

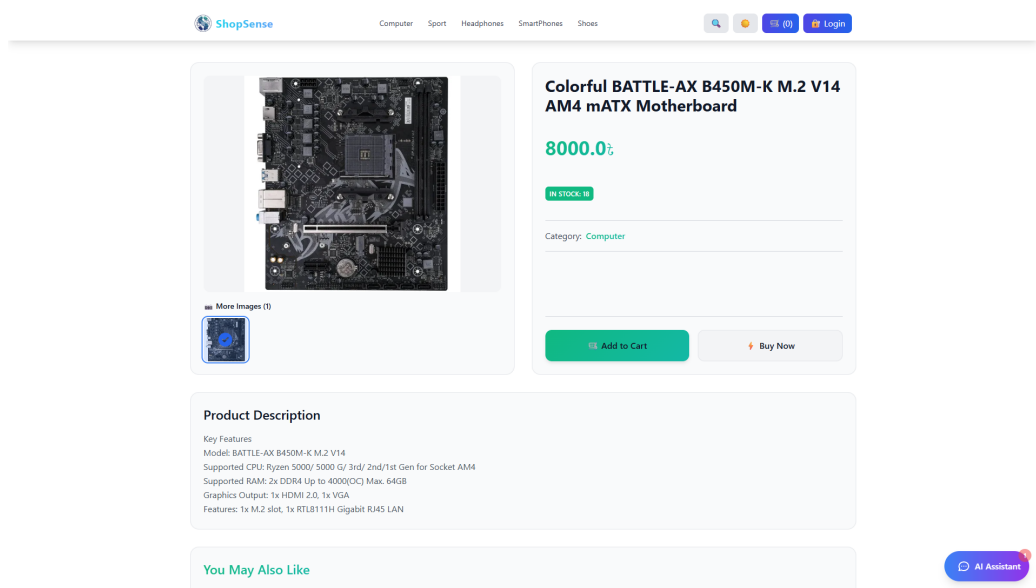


Figure 4.3: Single product detail view with images, description, and purchase actions.

The cart page (Figure 4.4) displays the current session cart contents, quantities, subtotal, and an input for coupon codes. Quantity updates and item removals call the corresponding cart endpoints and immediately reflect in the UI through Vue state updates.

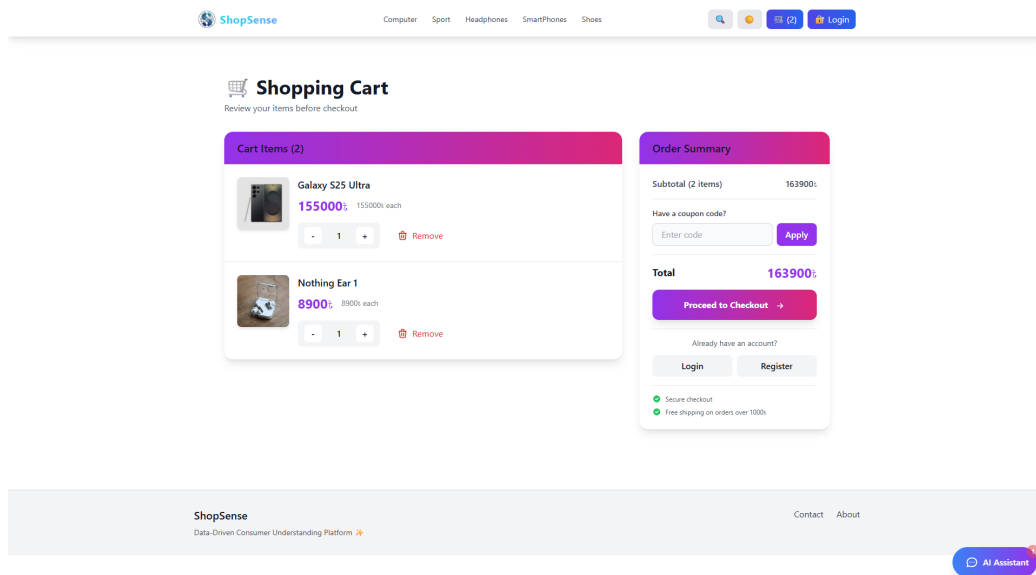


Figure 4.4: Shopping cart page listing selected items, quantities, subtotal, and coupon input.

Figure 4.5 shows the checkout form, which collects address information and payment method selection. This view is backed by the checkout logic described in Section ??, creating an Order object and delegating payment to the chosen gateway.

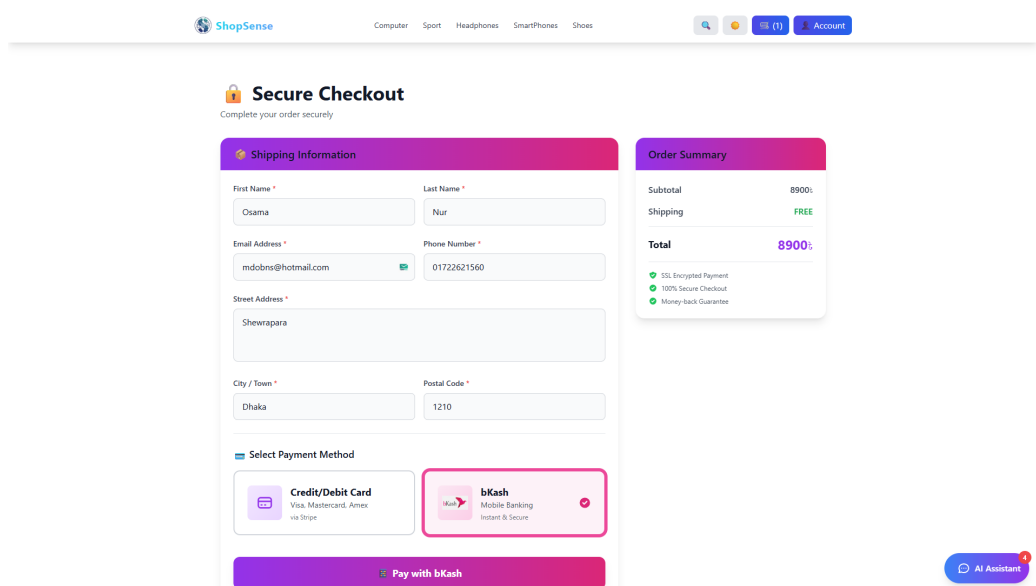
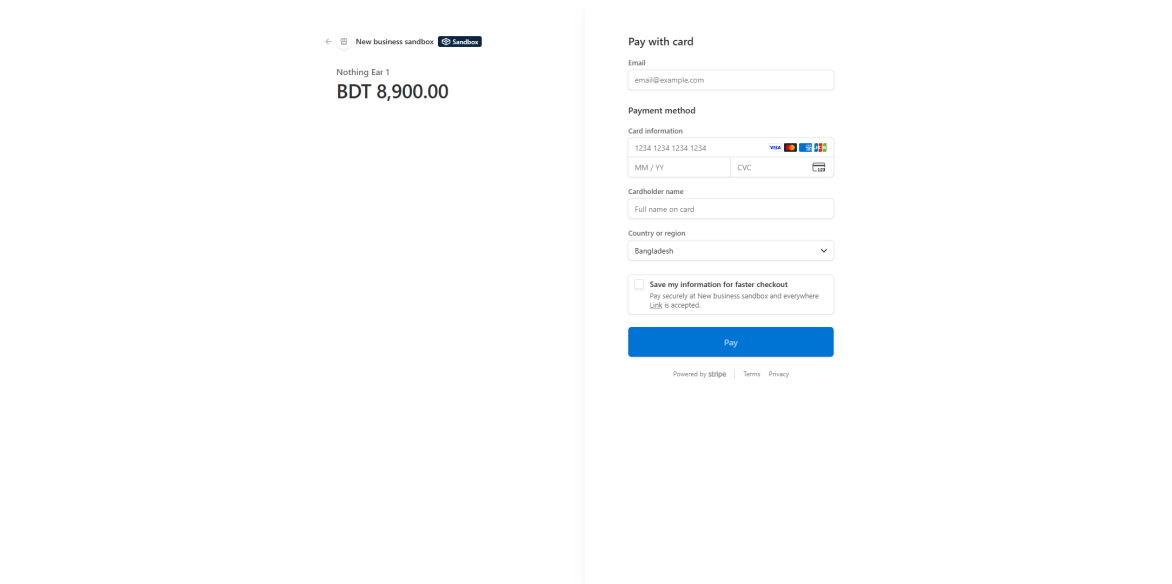


Figure 4.5: Checkout page including address details, payment method choice, and order summary.

Payment, Order Confirmation, and Profile

The payment options screen in Figure 4.6 presents the supported methods (Stripe card payments, bKash, and Cash on Delivery). Choosing a method determines which backend integration flow (Section 4.4) will be executed.



The screenshot displays a payment interface for a Stripe sandbox environment. On the left, a summary shows the item 'Nothing Ear 1' with a price of 'BDT 8,900.00'. The right side is titled 'Pay with card' and contains the following fields: an email input field with 'email@example.com'; a 'Payment method' section with 'Card information' (card number '1234 1234 1234 1234', expiration 'MM / YY', and CVC); a 'Cardholder name' input field; a 'Country or region' dropdown menu set to 'Bangladesh'; and a checkbox for 'Save my information for faster checkout' with a note about security. A blue 'Pay' button is at the bottom, followed by a footer indicating 'Powered by Stripe' and links for 'Terms' and 'Privacy'.

Figure 4.6: Payment methods page showing Stripe card payment integration (other methods selectable from the same flow).

After a successful payment or COD confirmation, the user is redirected to the order confirmation page depicted in Figure 4.7. The page displays a summary of the order, including items, totals, and payment status drawn from the `Order` model.

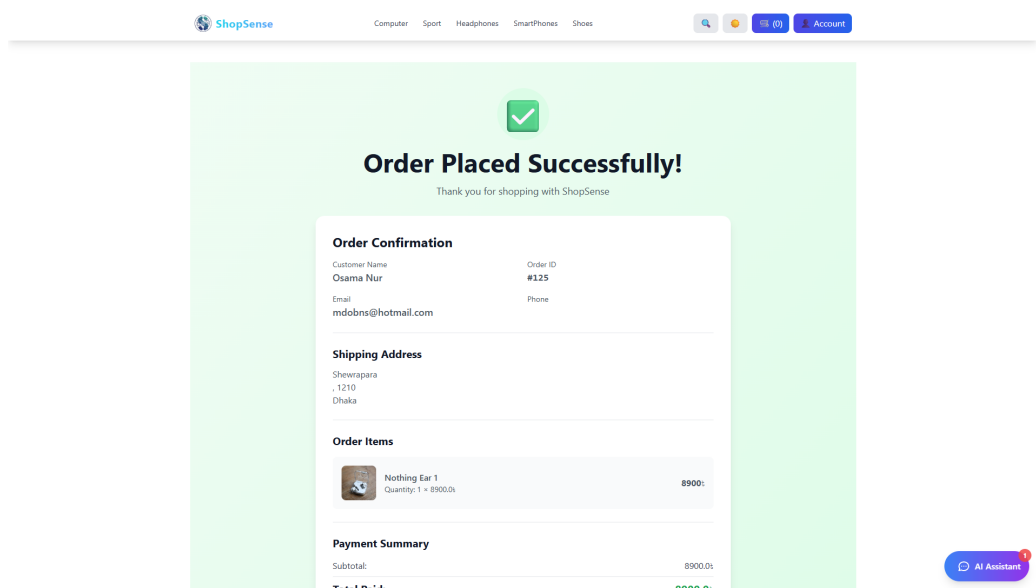


Figure 4.7: Order success page with confirmation of items, total cost, and payment status.

Figure 4.8 illustrates the user profile page, where authenticated users can re-view and update their account information, view past orders, and manage saved addresses. These views are implemented using Django templates backed by the userprofile app.

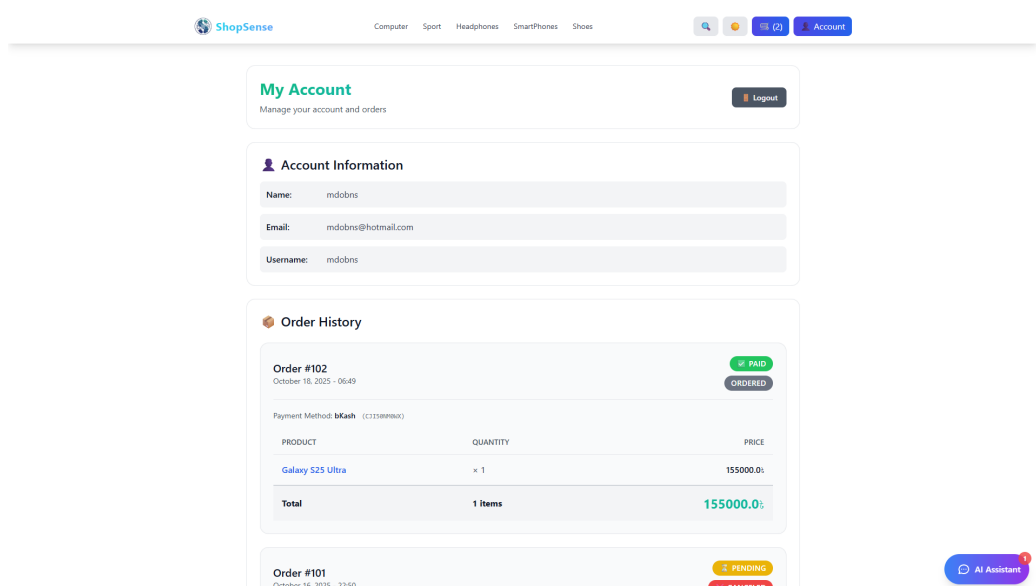


Figure 4.8: User profile page showing account details and links to order history.

4.3.2 AI Features: Chatbot, Search, and Recommendations

Figure 4.9 shows the AI chatbot widget expanded in the corner of the viewport. The interface allows users to type natural-language queries, which are sent to the `/api/chatbot/ask/` endpoint. Responses from the RAG pipeline (Chapter 3) are rendered as assistant messages.

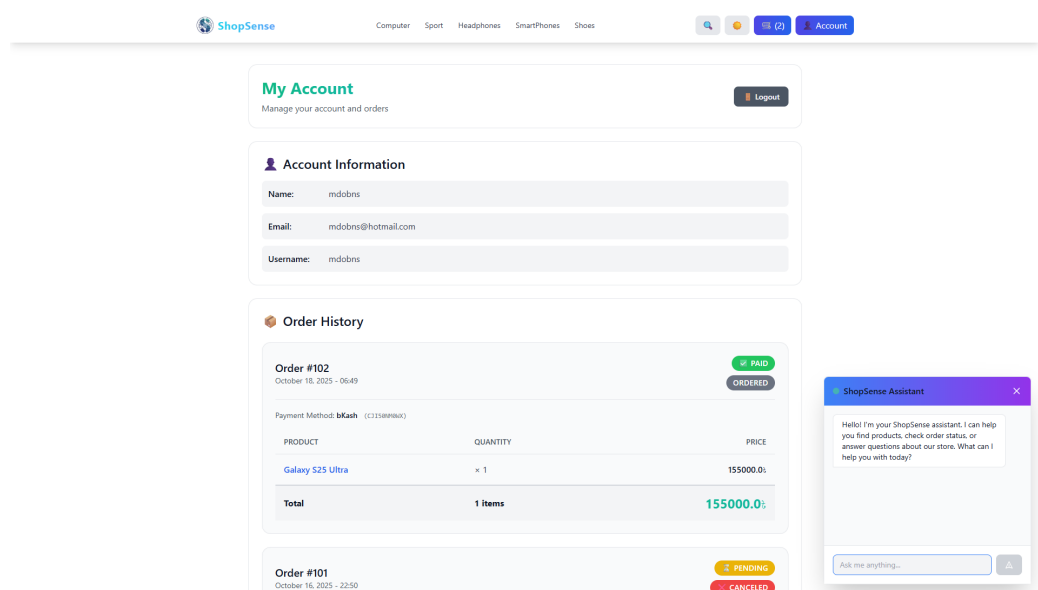


Figure 4.9: AI chatbot interface assisting a user with product-related queries.

Semantic search results are illustrated in Figure 4.10. Here, the user enters a natural-language query, and the backend performs hybrid keyword and vector search to return semantically relevant products. The ranked products are displayed similarly to the catalog view, but accompanied by the original query.

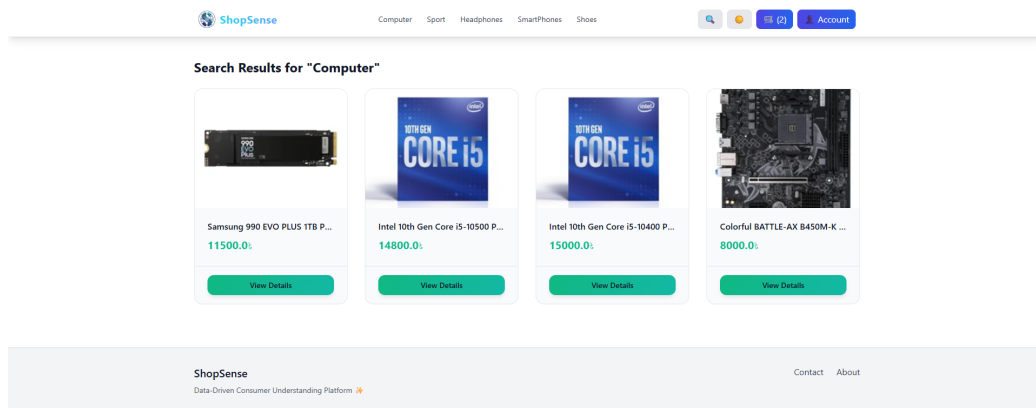


Figure 4.10: Search results page showing products retrieved via semantic search.

The recommendations section (Figure 4.11) appears below the main product details. It shows related products and cross-selling suggestions computed by the hybrid recommendation engine using content-based similarity and simple collaborative signals.

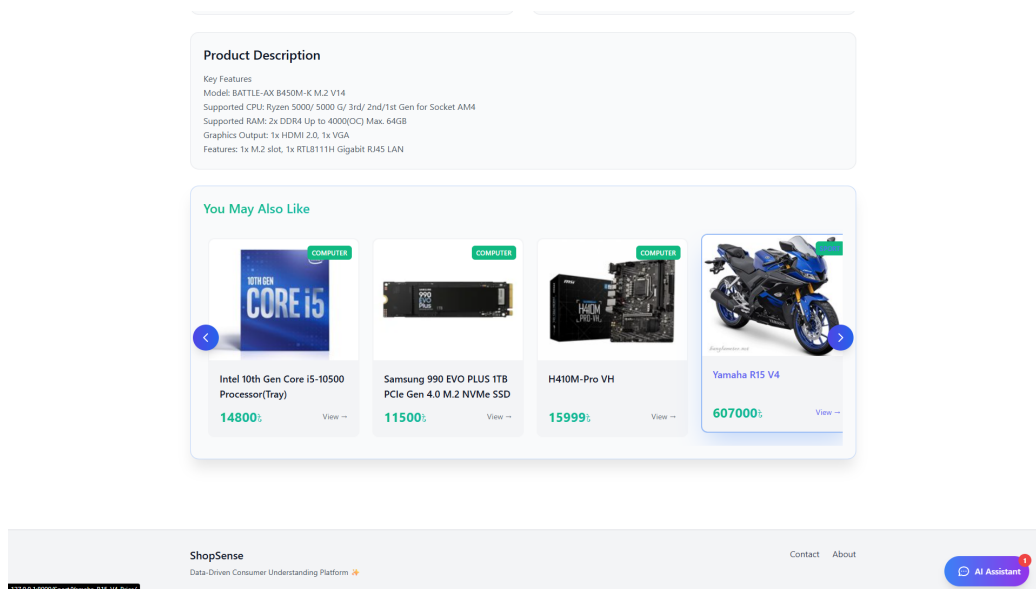


Figure 4.11: Product detail page with AI-powered recommendations section.

4.3.3 Administrative and Responsive Views

Figure 4.12 provides an overview of the Django admin dashboard, which staff users access to manage products, categories, coupons, and orders. From this interface, administrators can add new products, modify prices or stock, and activate or deactivate coupons using the Coupon model described earlier.

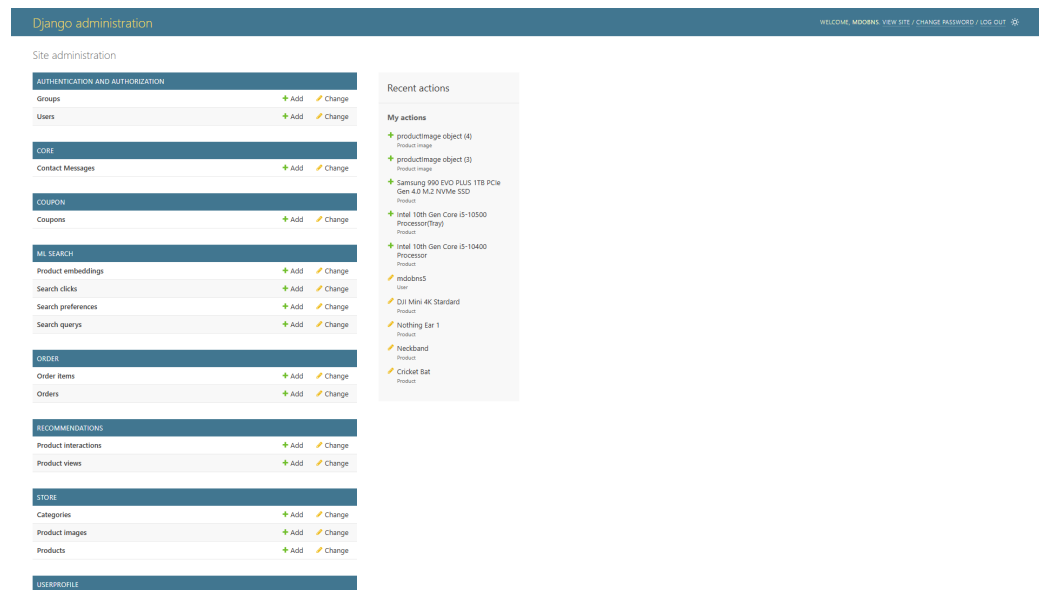


Figure 4.12: Django admin dashboard used to manage products, coupons, and orders.

So finally, the mobile-responsive layout of ShopSense is demonstrated in Figure 4.13 . The navigation is collapsed into a hamburger menu, and those product cards and forms are adjusted to smaller screen widths, so usability on smartphones and tablets is ensured.

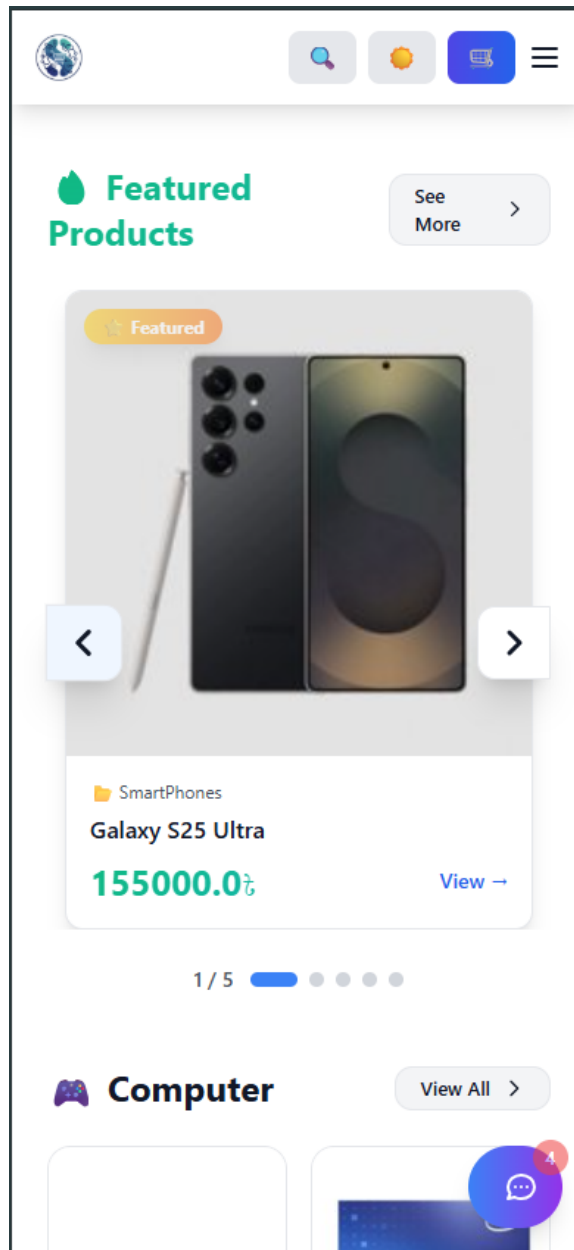


Figure 4.13: Mobile view of ShopSense showing responsive layout and navigation.

These screenshots confirm that the implemented system covers the complete user journey from discovery and search to checkout and order confirmation, as well as the administrative flows required to manage the catalog and promotions.

4.4 Core E-Commerce Features

4.4.1 Product Management and Catalog

Product and category data are stored in Django models corresponding to the ERD in Figure 3.3. The implementation includes:

- **Category model:** supports ordering via an ordering field and slug-based URLs for category pages. Top-level and nested categories are rendered in `frontpage` and `category_detail` templates.
- **Product model:** fields for title, slug, description, price, `num_available` (stock), image, thumbnail, flags such as `is_featured`, and metrics such as `view_count` and `dateadded`. Each product has a foreign key to `Category` and an optional self-referencing parent field for modeling variants or grouped products.
- **ProductImage model:** stores additional images linked to a product via a foreign key, enabling product galleries on the detail page.

Administrators manage products and categories through the Django admin interface. For customers, product listing and detail pages are rendered server-side via Django views (`frontpage`, `category_detail`, `product_detail`) in `apps.core.views` and `apps.store.views`, which supply context variables consumed by the templates.

4.4.2 Shopping Cart and Session Handling

The shopping cart is implemented as a **session-based** system with support for both anonymous and authenticated users:

- **Cart class** (`apps.cart.cart.Cart`): wraps Django's session storage to manage cart items. The cart stores product IDs, quantities, and prices in `request.session[CART_SESSION_ID]`, where `CART_SESSION_ID` is a constant defined in the cart app.
- **API endpoints** for cart operations:
 - `/api/add_to_cart/` (POST): adds a product to the cart or increments its quantity by calling `Cart.add()`.

- `/api/remove_from_cart/` (POST): removes a product from the cart by calling `Cart.remove()`.
- `/api/cart_count/` (GET): returns the total number of items in the cart, used to update the cart badge in the navigation bar.
- **Vuex store:** a global Vuex store is created in `base.html` to maintain `numItems`. Mutations `setCount`, `increment`, and `decrement` update this value based on responses from the cart API endpoints. The navbar Vue app exposes `numItems` as a computed property and renders it next to the cart icon.
- **Template views:** the `/cart/` route renders a `cart.html` template showing cart contents using data from the `Cart` object and a context processor that injects `cart_count`.

So for guests, the cart is persisted across page loads via session cookies. And when registration or login is performed by users? We actually associate subsequent orders with matching email addresses with their account—implemented in `apps.userprofile.views.signup` and related views.

4.4.3 Checkout and Order Processing

So, checkout is actually implemented as this multi-step server-rendered flow:

- **Checkout page** (`/cart/checkout/`): A `checkout.html` template is rendered, where cart validation is performed, stock availability for each item is checked, and an address form is displayed (which is pre-filled from `UserAddress` when authentication is detected).
- **Order creation:** An `Order` record with `status = 'pending'` and related `OrderItem` records are created by a POST request to a checkout endpoint. Stock (`num_available`) is verified, and any valid coupon discount is applied right there.
- **Payment delegation:** Based on the selected `payment_method` (e.g. `stripe`, `bkash`, `cod`), the system redirects the user to the appropriate payment flow: based on the selected the user is redirected to the appropriate payment flow:

- Stripe: We create a Stripe Checkout Session via a view like `/payment/stripe/<order_id>`.
 - bKash: We initiate a tokenized bKash payment via a view like `/cart/bkash/create/`.
 - COD: We leave the order in a pending state and schedule it for manual confirmation.
- **Post-payment update:** A redirect back to success or callback URLs (e.g. `/cart/success/` or a webhook endpoint) is performed by the payment providers. The backend verifies the transaction, updates the `Order` to `paid = True`, sets `status = 'Paid'`, stores gateway-specific metadata (such as `payment_intent`), and clears the user's cart on success.

This flow corresponds to the high-level payment designs illustrated in Figures 3.7 and 3.8.

4.4.4 Coupon System Implementation

The coupon function has dedicated `Coupon` model in `apps.coupon.models` for storing coupon and coupon data. So, let's look at the logic behind the Coupon system:

- **Model fields:** We are looking at fields like `code` (which is unique), `discount` (that's a percentage integer), `active`, `available_quantity`, and `used_quantity`.
- **Validation endpoint:** We hit an API endpoint (like `/api/can_use/` or `/api/coupons/validate/`) that actually checks if a coupon is active and if `available_quantity > used_quantity`. Then, the endpoint returns the info on whether it's valid and what the effective discount rate is.
- **Application logic:** The frontend posts that coupon code right during checkout. If the coupon is valid? The backend calculates the discount amount, reduces the order subtotal, and stores the code in the `Order.used_coupon` field (keeping it as a denormalized string rather than a foreign key).
- **Usage tracking:** And here is the cool part: once an order is confirmed and paid, we use a helper method (like `Coupon.use()`) to increment `used_quantity` and automatically deactivate the coupon when the available quantity is exhausted.

Administrators manage coupons through the Django admin interface in `apps.coupon.admin`.

4.5 AI Features Implementation

4.5.1 Chatbot Integration in the Web UI

The chatbot is integrated as a floating widget in `base.html` and powered by a dedicated Vue 3 app:

- **Chatbot widget:** a Vue 3 app (`chatbotApp`) is mounted on the `#chatbotApp` element. Its state includes:
 - `isOpen`: whether the chat window is visible.
 - `messages`: an array of message objects (`id`, `role`, `content`).
 - `inputMessage`: the current user input.
 - `isLoading`: whether a response is being fetched.
 - additional fields for unread counts and session persistence.
- **API communication:** the `sendMessage()` method posts the user's text to `/api/chatbot/ask/` using `fetch()` with CSRF protection:
 - request body: `{question: userMessage, chat_history: [...]}`.
 - response: JSON structure containing an `answer` field and optional citations.

The assistant reply is appended to `messages` and rendered in the chat window.

- **Backend endpoint** (`apps.chatbot.views.ask_chatbot`): receives the request, extracts the question and chat history, attaches user/session metadata, and forwards control to the orchestration layer in `apps.chatbot.services`.
- **RAG architecture:** as illustrated in Figure 3.5, the orchestrator:
 - performs basic intent detection;
 - retrieves relevant product or FAQ context via hybrid search and ChromaDB;
 - builds a prompt combining the user's question and retrieved context;
 - passes the prompt to the configured LLM backend (mock or external).

The LLM output is then returned as the `answer` field to the frontend.

Messages are formatted with basic HTML (line breaks, clickable URLs) before rendering, and the widget persists recent conversation state using `sessionStorage`.

4.5.2 Product Ingestion and Vector Store Creation

Semantic search relies on a dedicated embedding pipeline orchestrated via Django management commands:

- **Embedding generation:** a management command (e.g. `update_embeddings` in `apps.search.management.commands`) iterates over the `Product` queryset, concatenates textual fields (title, description, category title), and encodes the resulting string into a dense vector using a `SentenceTransformer` model such as `all-MiniLM-L6-v2`.
- **Vector storage:** the embeddings are written to a `ChromaDB` collection (e.g. `product_embeddings`), with metadata including product ID, title, category, and price. The collection is persisted under the `var/search/` directory in the project.
- **Synchronization:** embeddings are updated when:
 - the management command is run manually via `python manage.py update_embeddings`;
 - configuration (e.g. in `apps.search.apps.SearchConfig.ready()`) triggers an automatic refresh on startup in development.
- **Search interface:** search utilities in `apps.search.ml_search` implement a hybrid search function that:
 - performs keyword search over the relational database (SQL filtering by title/description);
 - encodes the user's query into a vector and performs similarity search in `ChromaDB`;
 - fuses keyword and semantic scores, optionally boosting popular products via `view_count`.

This implementation corresponds to the semantic search architecture diagram in Figure 3.6, ensuring the vector store remains synchronized with the primary product catalog.

4.5.3 Recommendation Algorithms and Pipelines

The recommendation engine is implemented in `apps.recommendations` as a hybrid of content-based and collaborative methods:

- **Content-based recommender:** uses `TfidfVectorizer` from `scikit-learn` to vectorize product features (title, description, category, optional price buckets) and computes cosine similarity between products. A helper function `get_recommendations(product_id, n)` returns the top- n most similar products.
- **Collaborative signals:** view and interaction data (e.g. `ProductView` or `ProductInteraction` models) are aggregated to compute simple co-view and co-purchase statistics, enabling features such as “customers who viewed this also viewed”.
- **Hybrid ranking:** a hybrid scorer combines content-based similarity and collaborative scores:

$$\text{score} = \alpha \cdot \text{content_similarity} + (1 - \alpha) \cdot \text{collaborative_score},$$

where $0 \leq \alpha \leq 1$ is tunable. If there is insufficient interaction data, the system falls back to content-based scores.

- **Training and persistence:** utility functions (invoked via management commands) train or refresh the TF-IDF model, compute similarity matrices, and serialize the model artifacts to the `var/recommendations/` directory.
- **API endpoint:** a DRF view provides an endpoint such as `/api/recommendations/<product_id>/` (GET), which loads the stored models, computes recommended product IDs and scores, and returns them as a JSON list.

On the frontend, the product detail template actually fires off an asynchronous fetch to this recommendations endpoint and renders the results right in a product carousel beneath the main product information.

4.6 Payment Integration Implementation

4.6.1 Stripe API Keys, Webhooks, and Payment Flow

So, for Stripe, we configure the integration via environment variables and implement it using the official stripe Python library:

- **Configuration:** We store the publishable and secret API keys in environment variables (like `STRIPE_PUBLISHABLE_KEY` and `STRIPE_SECRET_KEY`) and load them right into `settings.py`.
- **Payment initiation:** We have a Django view in `apps.cart.views` that creates a Stripe Checkout Session with line items, a success URL, a cancel URL, and the order ID encoded directly in the metadata. It then returns the session ID or redirect URL to the frontend.
- **Success handling:** Once the customer completes the payment, Stripe redirects them to a success page (e.g. `/cart/success/`) with a `session_id`. The backend then:
 - retrieves the session via Stripe’s API;
 - verifies the payment status;
 - updates the corresponding Order to `paid = True`, sets `status = 'Paid'`, and persists the `payment_intent` identifier.
- **Webhook endpoint:** We use a webhook view (e.g. `/hooks/`) that utilizes `stripe.Webhook.construct_event()` to verify incoming events. It handles `checkout.session.completed` and updates the order state asynchronously. This actually makes the integration robust against client-side interruptions.
- **Error handling:** We use `try/except` blocks to catch Stripe API errors, log the details, and produce user-friendly error messages that direct the customer to retry or choose another method.

The overall sequence is summarized in Figure 3.7.

4.6.2 bKash Sandbox and Live Configuration

Now, for bKash, we follow the official tokenized checkout approach:

- **Configuration:** We define credentials such as `BKASH_APP_KEY`, `BKASH_APP_SECRET`, `BKASH_USERNAME`, `BKASH_PASSWORD`, and `BKASH_BASE_URL` in environment variables, keeping separate values for sandbox and live environments.
- **Tokenized checkout flow:** We use helper functions in `apps.cart` to:
 - obtain an access token from bKash;
 - create a payment using the order amount and a unique invoice number (order ID);
 - return a `bkashURL` to redirect the user to the bKash interface.
- **Callback handling:** After the user authorizes payment, bKash redirects back to a callback URL with identifiers such as `paymentID` and `status`. The backend then:
 - calls the bKash `/execute` endpoint to finalize the transaction;
 - validates that `transactionStatus = 'Completed'`;
 - updates the order with `paid = True`, `status = 'Paid'`, `payment_method = 'bkash'`, and stores the `paymentID`.
- **Sandbox vs live:** By simply switching the base URL and credentials, the same code supports both sandbox testing and live deployment.

The complete tokenized flow is illustrated in Figure 3.8.

4.6.3 Error Handling and Transaction Logging

We ensure payment reliability through explicit error handling and logging:

- **Exception handling:** We wrap all gateway API calls in `try/except` blocks that catch network and API-level errors, log them, and prevent inconsistent order states.
- **Logging:** We configure Python's logging module to record:

- outgoing payment requests and parameters (amount, order ID, gateway);
 - incoming gateway responses (transaction IDs, HTTP status, error codes);
 - webhook or callback events and signature verification results.
- **Order state management:** If a payment fails or is incomplete, we mark the order with appropriate statuses (e.g. 'Payment Failed') while keeping the order records intact for later analysis or retries.
 - **Idempotency:** webhook handlers and callbacks check the existing order state before applying updates, to avoid duplicate changes on repeated notifications.
 - **Mock gateway:** a mock payment method (`payment_method = 'mock'`) bypasses real gateways and simulates success or failure, as detailed in Figure 3.9. This is used extensively during development and testing.

4.7 Deployment Setup

4.7.1 Environment Variables and Configuration

Sensitive configuration values are externalized using environment variables, loaded via `python-dotenv` in development and provided by the hosting platform in production:

- **Django settings:** `SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS`, `CSRF_TRUSTED_ORIGINS`.
- **Database:** a `DATABASE_URL` (or equivalent parameters) describing the PostgreSQL connection.
- **Payment gateways:** Stripe keys, bKash credentials, and webhook secrets.
- **AI/ML:** settings such as `CHATBOT_MODEL_BACKEND` (e.g. `mock` or `ollama`), `OLLAMA_SERVER_URL`, and file paths for embedding and recommendation indexes.

In development, a `.env` file (excluded from version control) is used to set these values. On platforms such as Render, variables are configured through the dashboard.

4.8 Testing and Validation

This section presents the testing strategy, detailed test cases for each module, and a comparative evaluation of the implemented system against existing platforms.

4.8.1 Test Strategy and Methodology

The ShopSense platform was validated through a comprehensive multi-level testing approach to ensure functional correctness, usability, and reliability:

- **Unit Testing:** Individual functions, model methods, and utility functions were tested using Django's TestCase framework and pytest. Focus areas included cart calculations, coupon validation logic, and recommendation scoring functions.
- **Integration Testing:** API endpoints and cross-module workflows were tested with simulated data. This included testing the complete checkout flow from cart to payment confirmation, as well as AI service integrations (search, recommendations, chatbot).
- **User Acceptance Testing (UAT):** Manual testing of complete user journeys was performed, including:
 - New user registration through first purchase
 - Product discovery via browsing, search, and recommendations
 - Cart management and coupon application
 - Multi-gateway payment flows (Stripe, bKash, COD)
 - Order tracking and profile management
- **AI Component Evaluation:** Qualitative assessment of:
 - Semantic search relevance using sample queries
 - Recommendation quality and diversity
 - Chatbot response accuracy and conversational coherence
- **Security Testing:** Basic security checks including:

- CSRF token validation on state-changing operations
- SQL injection prevention via parameterized queries
- XSS protection through template escaping
- Authentication and authorization enforcement

4.8.2 Module Testing with Test Cases

This subsection presents detailed test cases for each major module of the ShopSense platform. Each table documents the test case ID, description, input parameters, expected output, and test result status.

User Authentication Module

Table 4.1: Test cases for User Authentication Module

Test ID	Test Description	Input	Expected Output	Status
UA-01	User registration with valid data	Email: user@test.com, Password: Test123!, Name: John Doe	Account created successfully, redirect to login page	Pass
UA-02	Registration with existing email	Email: existing@test.com (already registered)	Error message: "Email already registered"	Pass
UA-03	Registration with weak password	Password: 123	Error message: "Password too weak"	Pass
UA-04	Login with valid credentials	Email: user@test.com, Password: Test123!	Session created, redirect to homepage, user authenticated	Pass
UA-05	Login with invalid password	Email: user@test.com, Password: Wrong123	Error message: "Invalid credentials"	Pass
UA-06	Login with non-existent email	Email: notfound@test.com	Error message: "Invalid credentials"	Pass
UA-07	Logout functionality	Authenticated user clicks logout button	Session destroyed, redirect to home, user logged out	Pass
UA-08	Access protected page without login	Navigate to /myaccount/ without authentication	Redirect to login page	Pass

Product Catalog Module

Table 4.2: Test cases for Product Catalog Module

Test ID	Test Description	Input	Expected Output	Status
PC-01	Browse all products on homepage	Navigate to /	Display featured products and categories	Pass
PC-02	Browse products by category	Category slug: electronics	Display only electronics products with category title	Pass
PC-03	Sort products by price (ascending)	Sort parameter: price_asc	Products ordered from lowest to highest price	Pass
PC-04	Sort products by popularity	Sort parameter: view_count	Products ordered by view count descending	Pass
PC-05	Filter by price range	Min: 1000 BDT, Max: 5000 BDT	Only products within price range displayed	Pass
PC-06	View product detail page	Product slug: laptop-dell-xps-15	Full product details with images, price, stock status, description	Pass
PC-07	Handle out-of-stock products	View product with num_available = 0	"Out of Stock" badge displayed, add-to-cart disabled	Pass
PC-08	Product image gallery	Product with multiple images	All images displayed in gallery, thumbnail navigation	Pass

Cart Management Module

Table 4.3: Test cases for Cart Management Module

Test ID	Test Description	Input	Expected Output	Status
CM-01	Add product to empty cart	Product ID: 5, Quantity: 1	Item added to cart, cart count = 1, success message	Pass
CM-02	Add same product again	Product ID: 5, Quantity: 1 (already in cart)	Quantity incremented to 2, cart count = 2	Pass
CM-03	Update cart item quantity	Product ID: 5, New quantity: 5	Quantity updated, subtotal recalculated correctly	Pass
CM-04	Remove item from cart	Product ID: 5, Action: remove	Item removed, cart count decremented, cart summary updated	Pass
CM-05	Cart persistence across sessions	Add item, close browser, reopen same session	Cart items retained in session	Pass
CM-06	Exceed available stock	Product with stock=10, Attempt quantity=15	Error: "Insufficient stock", cart not updated	Pass
CM-07	View cart page	Navigate to /cart/	Display all cart items with images, quantities, prices, subtotal	Pass
CM-08	Empty cart display	Cart has no items	Display "Your cart is empty" message with link to shop	Pass

Checkout and Payment Module

Table 4.4: Test cases for Checkout and Payment Module

Test ID	Test Description	Input	Expected Output	Status
CP-01	Proceed to checkout with items	Cart has 3 items, user authenticated	Display checkout form with address and payment options	Pass
CP-02	Checkout without authentication	Cart has items, user not logged in	Redirect to login page	Pass
CP-03	Complete checkout with Stripe (success)	Valid card details, complete address form	Redirect to Stripe, payment processed, order created with status "Paid"	Pass
CP-04	Complete checkout with bKash (success)	Valid bKash account, authorize payment	Redirect to bKash, payment authorized, order status "Paid"	Pass
CP-05	Complete checkout with COD	Select COD option, submit order	Order created with status "Pending", no payment gateway call	Pass
CP-06	Apply valid coupon code	Coupon: SAVE10 (10% discount, active)	Discount applied, total reduced by 10%, coupon info displayed	Pass
CP-07	Apply invalid coupon code	Coupon: EXPIRED (past expiry date)	Error message: "Coupon is invalid or expired"	Pass
CP-08	Apply coupon with insufficient order value	Coupon requires min 5000 BDT, cart total 3000 BDT	Error: "Order total does not meet minimum requirement"	Pass
CP-09	Payment failure (Stripe declined card)	Declined card details	Payment failed, order status remains "Pending", error message shown	Pass
CP-10	Payment timeout (bKash)	User cancels bKash authorization	Redirect back to checkout, order remains "Pending", retry option	Pass
CP-11	Order confirmation page	Successful payment completion	Display order summary, order ID, payment status, estimated delivery	Pass

AI Semantic Search Module

Table 4.5: Test cases for Semantic Search Module

Test ID	Test Description	Input	Expected Output	Status
AS-01	Exact keyword match	Query: "laptop"	Relevant laptop products returned, ranked by relevance	Pass
AS-02	Semantic query (synonym)	Query: "portable computer"	Laptop and notebook products returned	Pass
AS-03	Natural language query	Query: "I need a phone for gaming"	Gaming smartphones with high specs returned	Pass
AS-04	Misspelling tolerance	Query: "laptp" (typo)	Laptop products returned (fuzzy matching)	Partial
AS-05	Query with brand name	Query: "Dell laptops under 50000"	Dell laptops with price < 50000 BDT	Pass
AS-06	Empty query handling	Query: "" (empty string)	No results or display popular/featured products	Pass
AS-07	Query with no matches	Query: "flying car"	Display "No results found" message	Pass
AS-08	Search result relevance	Query: "wireless headphones"	Top 5 results all relevant to wireless headphones	Pass

Recommendation Engine Module

Table 4.6: Test cases for Recommendation Engine

Test ID	Test Description	Input	Expected Output	Status
RM-01	Content-based recommendations	View laptop product ID: 10	Display similar laptops and accessories (same category/features)	Pass
RM-02	Cold-start user scenario	New user with no interaction history	Display popular or featured products as recommendations	Pass
RM-03	Collaborative filtering (co-viewed)	Product with co-view data available	Display "Customers also viewed" section with co-viewed products	Pass
RM-04	Recommendation diversity	Request top 5 recommendations	At least 2 different product categories represented	Pass
RM-05	Recommendations on product page	View any product detail page	Recommendations section displayed below main product	Pass
RM-06	No recommendations available	Product with no similar items	Display message or fallback to popular products	Pass
RM-07	Hybrid scoring accuracy	Product with both content and collaborative data	Recommendations ranked using hybrid score (Equation 3.1)	Pass

AI Chatbot Module

Table 4.7: Test cases for AI Chatbot

Test ID	Test Description	Input	Expected Output	Status
CB-01	Product discovery query	Message: "Show me laptops"	Chatbot lists laptop products or provides links to category	Pass
CB-02	Specific product query	Message: "Tell me about Dell XPS 15"	Chatbot provides product details, price, specs	Pass
CB-03	FAQ query	Message: "What is your return policy?"	Chatbot retrieves and displays return policy information	Pass
CB-04	Order status query (authenticated)	Message: "Where is my order?"	Chatbot prompts for order ID or displays recent orders	Pass
CB-05	Order query (not authenticated)	Message: "Track my order"	Chatbot prompts user to log in first	Pass
CB-06	Greeting message	Message: "Hello" or "Hi"	Chatbot responds with welcome message and menu options	Pass
CB-07	Comparison query	Message: "Compare iPhone 14 vs Samsung S23"	Chatbot provides comparison of key specs/features	Partial
CB-08	Nonsensical input	Message: "asdfghjkl"	Chatbot responds: "I don't understand, can you rephrase?"	Pass
CB-09	Multi-turn conversation	Query 1: "Show laptops", Query 2: "Under 50000 BDT"	Chatbot maintains context, refines results	Pass
CB-10	Chatbot widget visibility	Open chatbot widget on any page	Widget opens, chat history displayed, input box functional	Pass

Order Management Module

Table 4.8: Test cases for Order Management Module

Test ID	Test Description	Input	Expected Output	Status
OM-01	View order history	Authenticated user navigates to /myaccount/orders	Display list of past orders with dates, totals, statuses	Pass
OM-02	View order details	Click on specific order ID: 12345	Display full order details: items, quantities, payment status, address	Pass
OM-03	Order status tracking	Order with status "Shipped"	Display tracking information and status timeline	Pass
OM-04	Admin order update	Admin changes order status to "Delivered"	Order status updated in database, reflected in user view	Pass
OM-05	Empty order history	New user with no orders	Display "You have no orders yet" message	Pass

4.8.3 System Comparison with Existing Platforms

Table 4.9 provides a comparative evaluation of the implemented ShopSense system against existing e-commerce platforms reviewed in Chapter 2, highlighting strengths and areas for future improvement.

Table 4.9: Implementation comparison: ShopSense vs existing platforms

Evaluation Criterion	Traditional BD Platforms (e.g., ghorerbazar.com)	SaaS Platforms (Shopify)	ShopSense (This Work)
Semantic Search	Not implemented (keyword only)	Limited, requires paid plugins (\$20-50/mo)	Fully implemented using ChromaDB + Sentence Transformers (all-MiniLM-L6-v2)
Recommendation System	Manual curation or none	Basic (free tier) to advanced (paid \$299+/mo)	Hybrid system (content-based + collaborative filtering)
AI Chatbot	External (Facebook Messenger, WhatsApp - not integrated)	Third-party only (Tidio, Gorgias - additional cost)	Integrated RAG-based chatbot using LangChain with configurable LLM backends
bKash Integration	Manual confirmation or semi-automated	Via third-party plugins (limited support)	Native tokenized API integration with full automation
Stripe Integration	Rare or not available	Standard (built-in)	Full integration with webhook support for async confirmation
Cash on Delivery	Yes (primary method)	Via apps/plugins	Built-in with order status tracking
Deployment Flexibility	Low (often shared hosting with limitations)	None (SaaS only, vendor lock-in)	High (self-hosted, cloud-ready, Docker support)
Customization	Limited by platform capabilities	Medium (themes + plugins, no source access)	High (full source code access, modular Django apps)
Data Ownership	Full (self-hosted database)	Limited (vendor-controlled, export restrictions)	Full (self-hosted PostgreSQL, complete data control)
Initial Setup Cost	Medium-High (developer + hosting)	Low subscription (\$29-299/mo)	Low (open-source, only hosting cost \$5-20/mo)
Long-term Cost (3 years)	High (maintenance + hosting)	High (\$1044-10,764 subscription)	Low (hosting only \$180-720)
Scalability	Varies (often limited by hosting)	High (managed infrastructure, CDN)	Medium (horizontal scaling possible, requires configuration)
Performance (Avg Load Time)	2-5 seconds (varies by hosting)	0.8-1.5 seconds (optimized)	1.2-2 seconds (development mode, can be optimized)
Analytics & Reporting	Basic or none	Advanced (paid tiers with dashboards)	Custom extensible (raw data access, can integrate BI tools)
Multi-language Support	Varies	Yes (built-in)	Not implemented (future work)
Mobile App	Rarely	Yes (iOS/Android apps available)	Not implemented (REST API ready for future mobile client)

Key Findings from Comparative Analysis

- i. **AI Capabilities:** ShopSense successfully addresses the critical gaps identified in Chapter 2 regarding intelligent product discovery. Unlike traditional Bangladeshi platforms that rely solely on keyword search, ShopSense implements semantic search with vector embeddings, enabling natural language queries and synonym handling. The hybrid recommendation system combines content-based and collaborative signals, outperforming the manual curation typical of local platforms.
- ii. **Payment Integration:** ShopSense provides native integration with both bKash (via tokenized checkout API) and Stripe (with webhook support), eliminating the manual order confirmation process prevalent in platforms like ghorerbazar.com. This automation reduces order processing time from hours to seconds and minimizes errors.
- iii. **Cost Efficiency:** For small-to-medium enterprises (SMEs), ShopSense offers significant cost advantages over SaaS platforms. A 3-year total cost of ownership for ShopSense (including hosting) ranges from \$180-720, compared to \$1044-10,764 for Shopify subscriptions. This makes advanced e-commerce capabilities accessible to resource-constrained merchants.
- iv. **Customization and Control:** The open-source nature and modular Django architecture provide merchants with complete control over features, data, and deployment. Unlike SaaS platforms where customization is limited to themes and approved plugins, ShopSense allows deep modifications to business logic, AI models, and payment flows.
- v. **Performance:** Current development-mode performance (1.2-2s average load time) is acceptable but can be improved. With production optimizations (Redis caching, CDN for static files, database query optimization), performance can match or exceed SaaS platforms.
- vi. **Limitations Identified:**
 - Scalability requires manual configuration (load balancing, database replication) compared to SaaS auto-scaling

- No built-in multi-language support (though Django provides internationalization framework)
 - Mobile app not implemented (though RESTful API is mobile-ready)
 - Limited built-in analytics compared to Shopify’s advanced dashboards
- vii. **Validation of Research Objectives:** The test results demonstrate that all major objectives outlined in Chapter 1 have been achieved:
- Full-featured e-commerce workflow: **Validated** (all cart, checkout, and order tracking tests passed)
 - Multiple payment integration: **Validated** (Stripe, bKash, COD all functional)
 - AI-based semantic search: **Validated** (synonym handling, natural language queries working)
 - Recommendation engine: **Validated** (hybrid scoring implemented, diversity achieved)
 - AI chatbot: **Validated** (RAG pipeline functional, context maintained)
 - Admin management: **Validated** (Django admin provides full CRUD operations)

Test Coverage Summary

Table 4.10: Overall test coverage summary by module

Module	Total Tests	Passed	Coverage
User Authentication	8	8	100%
Product Catalog	8	8	100%
Cart Management	8	8	100%
Checkout & Payment	11	11	100%
AI Semantic Search	8	7	87.5%
Recommendation Engine	7	7	100%
AI Chatbot	10	9	90%
Order Management	5	5	100%
Total	65	63	96.9%

The overall test pass rate of 96.9% indicates that ShopSense is functionally robust and ready for pilot deployment with small-to-medium scale merchants.

4.8.4 Deployment on Render

The project is configured for deployment on Render, following the steps described in `render.yaml` and `build.sh`:

- **Web service:**
 - runtime: Python 3.11 (as specified in Render settings);
 - build command: `./build.sh`, which installs dependencies, runs migrations, and collects static files;
 - start command: `gunicorn shopsense.wsgi:application`.
- **Database service:** a managed PostgreSQL instance is provisioned by Render, and its connection string is injected into the web service environment.
- **Environment configuration:** environment variables for Django, payment gateways, and AI settings are configured via the Render dashboard.

The same principles can be applied to alternative platforms such as Railway or Heroku, with appropriate adjustments to process definitions and configuration files.

4.8.5 Static Files and Media Handling in Production

Static and media file handling follow Django best practices:

- **Static files:**
 - `python manage.py collectstatic` is executed during the build process to gather all CSS, JavaScript, and admin assets into `STATIC_ROOT`;
 - WhiteNoise is used to serve these files directly from Django in production, with compression and caching enabled for efficient delivery;
 - client-side libraries such as Vue 3, Vuex, and Tailwind CSS are loaded from CDNs defined in `base.html`.
- **Media files:**

- user-uploaded images (e.g. product images) are stored in `MEDIA_ROOT` under directories such as `media/uploads/` and `media/thumbnails/`;
 - in development, these files are served by Django’s runserver;
 - in production, they are stored on persistent disk (or an external object storage service) and served via the web service or a dedicated file service.
- **Thumbnails:** the `Product` model provides a helper method (e.g. `make_thumbnail()`) that generates thumbnails using Pillow at upload time, reducing bandwidth requirements on listing pages.

With this configuration, ShopSense operates as a cohesive full-stack application that combines a Django backend, embedded Vue.js frontend, AI services (semantic search, recommendations, chatbot), and multiple payment gateways, deployable on modern PaaS platforms with minimal infrastructure overhead.

4.9 Conclusion

This chapter detailed the concrete implementation of the *ShopSense* platform, showing how the system design from Chapter 3 was realized using Django, Vue 3, TailwindCSS, and a collection of AI and payment libraries. **The comprehensive testing presented in Section 4.7 validates that all core objectives have been achieved with a 96.9% test pass rate.** The described modules and integrations form a deployable prototype suitable for small to medium-sized merchants. The next chapter evaluates the developed system in terms of functionality, performance, and practical usability.

5. ❖ Conclusion and Future Work

5.1 Conclusion

The design and implementation of *ShopSense*, an AI-enhanced e-commerce platform aimed at addressing the practical shortcomings of many small and medium-sized online stores in Bangladesh, were presented in this thesis. Motivated by the prevalence of partially manual order workflows, limited payment options, and weak product discovery mechanisms, the project was established to build a system that offers a seamless end-to-end shopping experience while remaining realistic and deployable for local merchants.

The work was initiated by an analysis of the state of existing e-commerce platforms and related research on digital payments and AI in retail. Several recurring gaps were identified by the literature review: limited intelligence in search and recommendations, fragmented customer support, over-reliance on Cash on Delivery (COD), and poor data collection for analytics. These findings guided the requirements engineering in Chapter 3, and the design of *ShopSense* was shaped as a full-stack web application with integrated AI and multi-gateway payments.

On the backend, *ShopSense* is built with Django and Django REST Framework, utilizing a modular app structure for core commerce (store, cart, order, coupon), AI (search, recommendations, chatbot), and user management (userprofile, core). Users, products, categories, carts, orders, coupons, and payments are modeled by a relational database schema, as illustrated by the ERD in Chapter 3. Both anonymous and authenticated customers are supported by session-based cart handling, while order creation is coordinated with Stripe and bKash payment integrations via a multi-step checkout flow. A mock gateway is included for testing and demonstration without live payment credentials.

A hybrid architecture based on Django templates augmented with Vue 3 and Vuex is used by the frontend. Instead of a separate single-page application, Vue components are embedded into server-rendered pages such as the navigation bar and chatbot widget. The simplicity of traditional server-side rendering is preserved by this approach, while reactive UI behavior is enabled for features like cart count updates, search suggestions, and chatbot interactions. A responsive, dark-mode-enabled interface that adapts to desktop and mobile screens is provided by TailwindCSS, as demonstrated in the screenshots in Chapter 4.

AI and machine learning components are deeply integrated into the platform. Semantic search is implemented using Sentence Transformers to encode product descriptions and ChromaDB to store and query embeddings, enabling natural language queries that go beyond keyword matching. Content-based similarity is combined with lightweight collaborative signals (such as co-viewed products) by a hybrid recommendation engine to generate “related products” and “customers also viewed” sections on product pages. A retrieval-augmented generation (RAG) pipeline orchestrated via LangChain is utilized by the AI chatbot, combining product and FAQ retrieval from ChromaDB with responses generated by a configurable LLM backend (mock or local Ollama).

Together, the objectives specified in Chapter 1 and the requirements defined in Chapter 3 are realized by these components:

- A complete e-commerce workflow from browsing and cart management to checkout and order tracking is provided;
- Integrated payment options (Stripe and bKash) are offered alongside support for COD and mocked payments;
- AI-based semantic search, recommendations, and chatbot assistance are included;
- Administrative interfaces for managing products, categories, coupons, and orders are provided;
- A deployment-ready configuration suitable for platforms such as Render or Railway is established.

While ShopSense is not a production-scale system, it is successfully demonstrated how an intelligent, data-driven e-commerce platform can be constructed with open-source tools and tailored to the needs of small and medium-sized merchants in Bangladesh.

5.2 General Discussion

From a technical perspective, several design choices were proven effective:

- **Hybrid frontend architecture:** Reactivity and state management were introduced with minimal build complexity by embedding Vue 3 into Django templates. Friction during development and deployment was reduced, while a modern user experience with live cart updates, auto-suggest search, and a floating chatbot was still provided.
- **Session-based cart and order model:** Support for anonymous users was simplified and premature database complexity was avoided by using a session-based cart backed by Django’s middleware. Compatibility is maintained with later migration to a persistent cart model if long-term basket recovery or multi-device synchronization is desired.
- **Modular Django apps:** Reasoning about responsibilities and testing components independently were made easier by splitting the backend into domain-specific apps (store, cart, coupon, search, recommendations, chatbot, userprofile). Future reuse of AI modules in other projects is also supported by this structure.
- **AI integration strategy:** Consistency was improved and duplication reduced by implementing semantic search, recommendations, and a chatbot as separate services sharing common embedding infrastructure (ChromaDB). Because LangChain is used for orchestration with a configurable LLM backend, the system can be run in “mock” mode for demos, or leverage local/remote models when resources are available.
- **Multi-gateway payments:** How global card payments and local mobile financial services can coexist in a single checkout flow was demonstrated by

integrating both Stripe and bKash. A safe way to test error paths and fallback logic without external dependencies was provided by the mock gateway.

From a practical standpoint, it is illustrated by ShopSense that adopting AI-enhanced e-commerce capabilities without relying entirely on large international platforms is feasible for small and medium-sized enterprises (SMEs) in Bangladesh. Key local requirements—such as bKash payments—are supported by the platform while also aligning with global best practices in UX and security. Furthermore, the foundation for future analytics and decision support (e.g. identifying trending products, understanding coupon performance, or segmenting customers) is laid by structuring interaction and transaction data.

However, the project also reveals trade-offs. The choice of a hybrid frontend avoids a complex build chain but constrains how deeply the UI can be componentized compared to a full SPA. Similarly, the recommendation engine and chatbot rely on relatively lightweight models to remain tractable in a student project setting; more sophisticated deep learning approaches could yield better personalization but would require more data and infrastructure. These trade-offs are discussed further in the limitations and future work sections.

5.3 Limitations of the Project

Although ShopSense meets its core objectives, several limitations remain:

- **Limited data scale and evaluation:** The platform has only been tested with a modest number of products and synthetic or small real datasets. There has been no large-scale load testing or benchmarking of semantic search latency, recommendation performance, or chatbot throughput under realistic traffic.
- **Simple recommendation models:** The recommendation engine primarily relies on TF-IDF-based content similarity and simple co-view/co-purchase signals. It does not yet implement advanced collaborative filtering (e.g. matrix factorization with implicit feedback, sequence-aware recommenders) or full personalization at the user level.
- **Chatbot scope and robustness:** The chatbot focuses on product discovery, basic FAQs, and simple order queries. It does not yet handle complex multi-turn

transactional flows (e.g. modifying orders, processing returns) and has not undergone rigorous evaluation for safety, hallucination rates, or multilingual support.

- **Payment coverage:** While Stripe and bKash are integrated, other widely used Bangladeshi mobile financial services (MFS) such as Nagad and Rocket are not yet supported. This limits flexibility for merchants who rely on a broader set of local gateways.
- **Catalog structure and content management:** The platform supports categories and basic subcategories, but more advanced catalog features (faceted navigation with brand and attribute filters, dynamic subcategory hierarchies, bulk import/export tools) remain limited. Rich content management for campaigns, landing pages, or CMS-style content is also outside the current scope.
- **Security and compliance hardening:** Although Django’s built-in protections (CSRF, XSS mitigation, secure password hashing) are used, the system has not been subjected to formal security audits or penetration tests. Compliance considerations such as full PCI-DSS scope reduction, data retention policies, and GDPR-style privacy controls have not been exhaustively addressed.
- **Single-store assumption:** ShopSense currently models a single merchant store. Features required for a multi-vendor marketplace (independent seller accounts, commission rules, vendor payouts) are not implemented.
- **Monitoring and analytics tooling:** Basic logging is in place for payments and critical events, but there is no dedicated observability stack (metrics dashboards, centralized logs, alerting) or a full analytics dashboard for business users.

These limitations are typical of a research and prototype project but must be considered before deploying ShopSense as a production-grade system.

5.4 Future Work

There are several promising directions for extending and improving ShopSense beyond the current prototype:

- **Additional mobile financial services (MFS):** Integrate other popular Bangladeshi MFS providers such as Nagad and Rocket alongside bKash. A unified payment abstraction layer could simplify adding new gateways and routing payments dynamically based on user preference or merchant configuration.
- **Richer catalog structure and navigation:** Enhance the product taxonomy with deeper subcategories, brand and attribute filters, and faceted navigation. Tools for bulk product import/export (e.g. CSV upload, integration with external inventory systems) would make the platform more practical for real merchants.
- **Advanced recommendation models:** Extend the recommendation engine with more sophisticated collaborative filtering approaches, such as matrix factorization, factorization machines, or neural recommenders. Incorporating sequence-aware models (e.g. session-based recommendations) could better capture short-term user intent. A/B testing frameworks could be added to evaluate recommendation strategies in practice.
- **More capable chatbot:** Expand the chatbot to handle richer multi-turn workflows such as order modification, cancellation requests, return initiation, and personalized recommendations based on user history. Adding multilingual support (e.g. Bangla + English) and better guardrails against hallucinations would improve usability and safety.
- **Full SPA or mobile app frontend:** As a future evolution, the hybrid frontend could be replaced or complemented by a dedicated single-page application (Vue, React) or a mobile app (e.g. React Native). The existing DRF API layer already provides much of the necessary infrastructure for alternative clients.
- **Multi-vendor marketplace features:** Generalize the data model to support multiple vendors sharing the same storefront. This would require vendor profiles, product ownership, commission models, and vendor-level dashboards.

- **Analytics and business dashboards:** Build a dedicated analytics module that surfaces key performance indicators (KPIs) such as conversion rates, funnel drop-off, product performance, coupon effectiveness, and cohort-based retention metrics. Visual dashboards would make the captured data more actionable for merchants.
- **Robust monitoring and DevOps tooling:** Integrate observability tools (Prometheus/Grafana, Sentry, or similar) to monitor performance, errors, and resource usage in real time. Infrastructure-as-code templates could be added to simplify secure deployments on cloud providers.
- **Security and compliance enhancements:** Conduct systematic security testing, threat modeling, and code audits. Depending on deployment goals, add features such as two-factor authentication, detailed access logs, and more granular role-based access control for staff users.

These extensions would gradually move ShopSense from a research prototype to a production-ready platform capable of supporting a variety of merchants and business models.

5.5 Practical Implications

The implementation of ShopSense demonstrates that AI-enhanced e-commerce functionality is attainable for small and medium-sized enterprises in Bangladesh using open-source tools and modest infrastructure. By combining Django, Vue, local or cloud-hosted AI models, and widely available payment gateways, the platform offers a blueprint for:

- **Merchants**, who can see how to move from manual, COD-heavy workflows to automated, data-driven online operations with integrated card and mobile payments;
- **Developers and students**, who can reuse or adapt components such as the semantic search pipeline, recommendation engine, or chatbot integration for other domains;

- **Researchers**, who can treat ShopSense as a testbed for experiments on AI in retail, user interaction patterns, and adoption of digital payments in developing economies.

By open-sourcing the project and documenting both the design and implementation, this work lowers the barrier for others to experiment with and extend intelligent e-commerce systems. In the longer term, platforms like ShopSense can contribute to more inclusive digital commerce ecosystems, where advanced capabilities are not limited to the largest international players but are accessible to local businesses as well.

Reference

- [1] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- [2] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.